VERIQTA

What Senior DevOps Interviews Actually Test:

40 Real Scenarios



REAL SCENARIOS

PRODUCTION THINKING

SENIOR ENGINEERING ANSWERS

ARCHITECTURE TRADE-OFFS

Instagram GitHub LinkedIn Telegram X @veriqta

Linux Production Troubleshooting

VERIQTA

Scenario 1

A production server reports 95% CPU utilization, but application traffic has not increased. How do you identify the actual source of contention?

1. Key Areas to Investigate

- Which processes are using CPU
- User space vs kernel space
- Run queue and load average
- Single thread vs multi thread
- Container or host level issue
- CPU limits (cgroups)
- Recent changes (deployments, cron jobs)
- System interrupts or kernel threads
- I/O wait or other hidden bottlenecks

2. Useful Commands

```
$ top                       # real-time view
$ htop                      # interactive view
$ ps -eo pid,ppid,cmd,%cpu \\
  --sort=-%cpu | head
$ pidstat 1                 # per-process CPU
$ vmstat 1                  # CPU, memory, I/O
$ sar -u 1 5                # CPU history
$ perf top                  # kernel/user CPU
$ dmesg | tail              # kernel messages
$ cat /proc/loadavg         # load average
$ docker stats              # if using containers
$ crontab -l                # check cron jobs
```

3. What to Look For

- High CPU process or zombie processes
- High system CPU (kernel space)
- CPU spikes from cron jobs or backups
- Runaway loop or misbehaving service
- Container using all CPU
- High number of interrupts (e.g. network)
- Resource limits causing throttling
- Unexpected process after deployment

4. Production Fixes (Examples)

- Kill or restart misbehaving process
- Fix code or configuration causing loop
- Adjust CPU limits or requests
- Disable or reschedule heavy cron job
- Investigate kernel or driver issues
- Roll back recent deployment if needed

Scenario 2

A Linux application becomes unresponsive despite available memory. How would you investigate process states, I/O wait, file descriptors, and kernel behavior?

1. Investigation Steps

1. Check if the process is running or stuck
2. Examine process state (R, S, D, Z, etc.)
3. Check CPU, memory, and I/O wait
4. Check open file descriptors
5. Identify threads and stack traces
6. Check disk I/O and filesystem health
7. Review kernel logs for errors
8. Look for deadlocks or external dependencies

2. Useful Commands

```
$ ps -eo pid,state,cmd | grep <process>
$ top                          # check process state
$ htop                         # thread view
$ vmstat 1                     # check I/O wait (wa)
$ iostat -xz 1                 # disk I/O analysis
$ pidstat -d 1                 # per-process I/O
$ lsof -p <pid> | wc -l        # count open files
$ lsof -p <pid> | head         # view open files
$ strace -p <pid>              # see system calls
$ dmesg | tail                 # kernel messages
$ cat /proc/<pid>/stack        # kernel stack (if stuck)
```

3. Process State Meanings

R	Running (using CPU)
S	Sleeping (waiting)
D	Uninterruptible sleep (usually I/O)
Z	Zombie (terminated)
T	Stopped (debug/trace)

4. Common Causes

- High disk I/O or slow storage
- NFS or network filesystem issues
- Blocked on external dependency
- Too many open file descriptors
- Deadlock or thread contention
- Kernel or driver issue
- File system full (inodes)
- Stuck in uninterruptible I/O (D state)

5. What to Check in I/O Wait

- Disk latency and queue depth
- Slow EBS/NFS or network storage
- Large log writes or database I/O
- filesystem errors
- High number of blocked processes
- Storage saturation
- Recent changes or heavy jobs

6. Production Fixes

- Restart stuck process (if safe)
- Fix storage or network issue
- Increase file descriptor limit (ulimit)
- Optimize disk I/O and logging
- Fix code causing deadlock
- Resolve external dependency
- Apply kernel or driver update
- Roll back recent change

VERIQTA

Memory Pressure and Resource Exhaustion

Scenario 3

A critical application is repeatedly terminated by the Linux OOM killer. How do you distinguish memory leaks, cgroup limits, and system-wide memory pressure?

1. What to Check First

1. **Confirm it's an OOM kill**
   Check kernel logs and container/runtime events.
2. **Determine the scope**
   Process only, container/cgroup, or system-wide.
3. **Analyze memory usage**
   Check trends, not just current usage.
4. **Check cgroup limits (if containerized)**
   Compare memory usage vs limits.
5. **Look for memory leaks**
   Analyze process memory growth over time.

2. Useful Commands

```
$ dmesg | grep -i oom                    # OOM killer logs
$ journalctl -k | grep -i oom            # kernel logs
$ top                                    # live memory usage
$ htop                                   # interactive view
$ free -h                                # system memory
$ vmstat 1                               # memory, swap, pressure
$ cat /proc/meminfo                      # detailed memory info
$ ps aux --sort=-%mem | head             # top memory processes
$ cat /sys/fs/cgroup/memory.max          # cgroup memory limit (v2)
$ cat /sys/fs/cgroup/memory/memory.limit_in_bytes
                                         # cgroup limit (v1)

$ pmap -x <pid> | head                   # process memory usage
$ smem -r                                # memory breakdown
                                           (if available)
```

3. Key Evidence and How to Interpret

- OOM killer log shows the killed process, memory usage and cgroup path.
- If only one process grows over time → likely a memory leak.
- If container limit is reached but host has free memory → cgroup limit.
- If host memory and swap are exhausted → system-wide memory pressure.
- Check for increasing RSS, heap usage or unreleased memory.

4. Common Causes

- Memory leak in application.
- Container/cgroup memory limit too low.
- System-wide memory pressure.
- Large cache usage (page cache).
- Many processes or high memory workloads.
- Kernel memory (slab) growth.

5. Production Fixes

- Fix the memory leak in code.
- Increase cgroup memory limit if needed.
- Optimize memory usage and enable proper resource requests/limits.
- Reduce memory pressure (free unused workloads, clear cache if safe).

6. Prevention

- Set realistic memory requests and limits.
- Monitor memory usage and OOM events.
- Use alerts for memory pressure and cgroup OOMs.
- Analyze memory growth trends.
- Use proper garbage collection and profiling.

Scenario 4

A server has sufficient disk space but applications cannot create new files. What evidence would you examine before changing anything?

1. Key Areas to Investigate

1. Verify actual disk space and inodes.
2. Check inode usage (not just disk space).
3. Check filesystem type and mount options.
4. Check user/file permissions and quotas.
5. Check if filesystem is read-only.
6. Check open file descriptor limits.
7. Check for filesystem errors or corruption.
8. Check for process or application limits.

2. Useful Commands

```
$ df -h                                  # disk space usage
$ df -i                                  # inode usage
$ lsblk                                  # block devices and mounts
$ mount | grep ' on '                    # mount options
$ findmnt-o TARGET,SOURCE,FSTYPE,OPTIONS # filesystem details
$ touch /path/testfile                   # test file creation
$ ls -ld /path                           # check permissions
$ getfacl /path                          # check ACLs (if used)
$ ulimit -n                              # open file limit (current)
$ cat /proc/sys/fs/file-max              # system file descriptor limit
$ lsof | wc -l                           # current open files
$ dmesg | tail                           # filesystem/kernel errors
$ journalctl -xe                         # recent system logs
```

3. Common Causes

- Inode exhaustion (no free inodes).
- Filesystem mounted read-only.
- User or directory permission issues.
- Disk quota limits (user or project quota).
- Too many open file descriptors.
- Filesystem errors or corruption.
- Application or process-level limits.
- Network filesystem issue (NFS/EFS).
- Read-only container filesystem.

4. How to Confirm

- `df -i` shows 100% inode usage.
- `mount` shows ro (read-only).
- `touch` fails with "Permission denied".
- `touch` fails with "No space left on device". but `df -h` shows free space → inode issue.
- `ulimit -n` or `lsof` shows high file handles.
- `dmesg` shows filesystem errors.
- Check NFS/EFS server or network issues.

5. Production Fixes

- If inode full: delete old files or increase inode count.
- If read-only: remount read-write after fixing underlying issue.
- Fix permissions or quotas.
- Close unused file descriptors and increase limits if needed.

6. Prevention

- Monitor inode usage (`df -i`).
- Set up alerts for high inode and disk usage.
- Use log rotation and cleanup policies.
- Monitor file descriptor usage.
- Regularly check filesystem health.
- Capacity planning for logs and temporary files.

Networking and DNS Failures

VERIQTA

Scenario 5

Services intermittently fail to communicate, but network connectivity tests succeed. How do you investigate DNS, connection pools, timeouts, and TCP behavior?

1. Investigation Approach

1. Confirm the exact error and pattern (intermittent, region, specific service).
2. Check recent changes (deployments, DNS, network, load balancer).
3. Test name resolution, not just IP connectivity.
4. Inspect connection pools, timeouts, retries and limits.
5. Examine TCP behavior (resets, timeouts, half-open connections).
6. Check server-side logs and metrics for connection errors.

2. Useful Commands

```
$ nslookup service.example.com          # basic DNS lookup
$ dig service.example.com               # detailed DNS
$ dig +trace service.example.com        # DNS path
$ getent hosts service.example.com      # system resolver
$ ping service.example.com              # basic connectivity
$ curl -v https://service.example.com   # TLS and headers
$ nc -vz service.example.com 443        # port test
$ telnet service.example.com 443        # TCP connectivity
$ ss -tulnp | grep :443                 # open sockets
$ ss -s                                 # connection summary
$ netstat -antp | grep ESTABLISHED      # active connections
$ tcpdump -nn host service.example.com  # capture packets
```

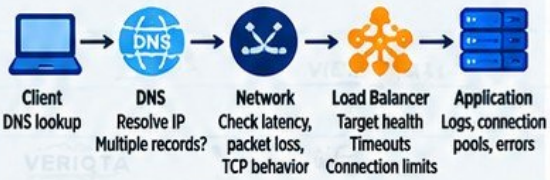
3. What to Check

- DNS resolution time and variability
- Multiple A/AAAA records and failover
- DNS TTL and cache behavior
- Connection pool exhaustion
- Timeout and retry settings
- TCP resets, retransmissions, SYN issues
- Load balancer health and target status
- Firewall or security group rules
- Ephemeral port exhaustion
- Server-side resource limits
- Application logs for connection errors

4. Common Causes

- Intermittent DNS resolution or stale cache
- Connection pool exhaustion
- Short timeouts or aggressive retries
- Load balancer target flapping
- Firewall dropping idle connections
- Ephemeral port exhaustion
- Server reaching connection limits
- Network path instability or packet loss

5. Example Investigation Flow



6. Fixes and Prevention

- Use reliable DNS (multi-AZ) and monitor resolution time
- Set appropriate connection pool sizes and timeouts
- Implement retry with exponential backoff
- Increase load balancer idle timeout if needed
- Monitor TCP resets and connection errors
- Tune OS limits (file descriptors, ephemeral ports)
- Add DNS and network observability (latency, failures)

Scenario 6

Users experience high application latency across multiple regions. How do you isolate network, load balancer, TLS, and application delays?

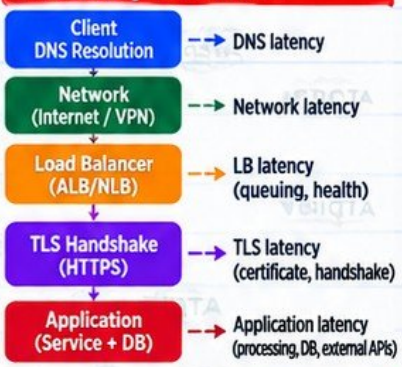
1. Investigation Steps

1. Reproduce from multiple regions using external and internal tests.
2. Measure latency at each layer (Client → Network → LB → App).
3. Identify where latency is introduced.
4. Check DNS, routing and network path.
5. Analyze load balancer metrics and logs.
6. Examine TLS handshake time.
7. Inspect application and database latency.
8. Compare results across healthy ys slow regions.

2. Useful Commands

```
$ dig service.example.com                # check DNS regionally
$ mtr -rw service.example.com            # network path and latency
$ ping -c 10 service.example.com         # basic latency
$ curl -w "time_namelookup:%{time_namelookup}\n" \
    -w "time_connect:%{time_connect}\n" \
    -w "time_appconnect:%{time_appconnect}\n" \
    -w "time_starttransfer:%{time_starttransfer}\n" \
    -w "time_total:%{time_total}\n" https://service.example.com
$ openssl s_client -connect service.example.com:443 \
  -servrvenane service.example.com       # TLS handshake
$ traceroute service.example.com         # network hops
$ tcpdump -nn host service.example.com # deeper analysis (if needed)
```

3. Latency Breakdown



4. What to Check

- DNS resolution time and correct region
- Network latency and packet loss
- Load balancer target health and cross-region routing
- TLS handshake duration and retries
- Application logs (slow queries, external API calls)
- Database latency and connection pool usage
- Error rates and retries increasing latency

5. Common Causes

- DNS routing to the wrong region
- Suboptimal network path or congestion
- Load balancer in the wrong region or unhealthy targets
- TLS handshake delays (large certificates, OCSP, CRL)
- Application or database slowness
- Cross-region dependencies
- Increased retries or timeouts
- Resource saturation (CPU, memory, DB, external APIs)

6. Fixes and Prevention

- Use geo-routing or latency-based DNS
- Deploy services closer to users (multi-region)
- Optimize load balancer configuration
- Use TLS session reuse and modern ciphers
- Monitor end-to-end latency with tracing
- Set proper timeouts and retry policies
- Scale application and database as needed
- Remove unnecessary cross-region calls

Docker and Container Runtime Failures

VERIQTA

Scenario 7

A container works locally but crashes repeatedly in production. How do you investigate runtime differences, entrypoints, permissions, and resource constraints?

1. Key Investigation Areas

1. Compare environment differences (env vars, config, secrets)
2. Check container logs and exit code
3. Verify entrypoint and command
4. Check file permissions and user (root vs non-root)
5. Compare resource limits (CPU, memory, PID, ulimits)
6. Check mounted volumes (config, data, permissions)
7. Review runtime differences (Docker vs Kubernetes)
8. Inspect health checks and restarts

2. Useful Commands

```
$ docker logs <container>          # view logs
$ docker ps -a                     # check exit code
$ docker inspect <container>       # full config
$ docker inspect --format
  '{{.Config.Entrypoint}}' <container>
$ docker inspect --format
  '{{.Config.Cmd}}' <container>
$ docker exec -it <container> sh   # debug inside
$ docker inspect --format
  '{{.Config.User}}' <container>
$ docker stats <container>         # live resource usage
$ docker run --rm -it --entrypoint sh <image>
$ cat /etc/os-release              # check base image
$ ls -l /app                       # check permissions
```

3. Common Causes

- Different environment variables
- Missing secrets or configuration
- Incorrect entrypoint or command
- File permission issues (non-root user)
- Missing dependencies (installed locally)
- Resource limits (OOM, CPU) in production
- Volume mount issues or read-only FS
- Architecture differences (amd64 vs arm64)
- Health check killing the container
- Application assumes local network/service

4. Fixes and Prevention

- Align environment variables and configs
- Use explicit entrypoint and command
- Ensure correct file permissions and user
- Set proper resource requests and limits
- Use the same base image and architecture
- Validate volume mounts and dependencies
- Improve logging and health checks
- Replicate production environment locally

Scenario 8

Containerized services experience unpredictable performance degradation. How would you distinguish CPU throttling, memory pressure, storage latency, and noisy-neighbor contention?

1. Investigation Steps

1. Identify the symptoms (high latency, slow response, timeouts)
2. Check container resource usage (CPU, memory, I/O, network)
3. Compare with node-level metrics
4. Look for throttling, OOM events, or I/O wait
5. Analyze application and system logs
6. Correlate with other workloads on the node
7. Check storage performance (disk/volume)
8. Review Kubernetes/Docker limits and QoS

2. Useful Commands

```
$ docker stats <container>          # real-time usage
$ docker inspect <container>        # limits and config
$ cat /sys/fs/cgroup/cpu/*/cpu.stat    # CPU throttling
$ cat /sys/fs/cgroup/memory/*/memory.stat # memory usage
$ dmesg | tail                      # OOM or kernel events
$ top                               # host-level view
$ iostat -xz 1                      # disk I/O stats
$ pidstat -d 1                      # per-process I/O
$ sar -u 1 5                        # CPU metrics
$ sar -r 1 5                        # memory metrics
$ sar -d 1 5                        # disk metrics
$ kubectl top pod <pod>             # K8s resource usage
$ kubectl describe pod <pod> # events and limits
$ kubectl get nodes -o wide  # node resource status
```

3. How to Distinguish the Cause

CPU Throttling
- High CPU usage with low actual CPU time
- High throttling in cpu.stat
- High load average, slow processing

Memory Pressure
- High memory usage
- OOMKilled events in logs
- Increased page faults and swapping

Storage Latency
- High I/O wait (wa)
- Slow disk or network storage
- High latency in iostat or cloud metrics

Noisy Neighbor Contention
- Other containers using high resources
- Host-level metrics show saturation
- Performance improves when node is quiet

4. Key Signals to Check

- CPU throttling metrics (cgroup cpu.stat)
- Memory usage, page faults, OOM events
- Disk I/O latency and I/O wait
- Network latency and packet loss
- Node-level CPU, memory, and disk usage
- Container restarts and kernel logs
- Other workloads on the same node
- Application metrics (latency, error rate, throughput)

5. Production Fixes

- Set proper CPU and memory requests and limits
- Use appropriate storage class and volume type
- Isolate noisy workloads using node pools or taints
- Monitor and alert on throttling, OOM, and I/O latency
- Use observability (Prometheus, Grafana)
- Optimize application performance and caching
- Use Horizontal Pod Autoscaler (HPA) where appropriate
- Right-size instances and reduce resource contention

6. Prevention Best Practices

- Set realistic resource requests and limits
- Use dedicated nodes for critical workloads
- Monitor cgroup metrics and node saturation
- Use QoS classes (Guaranteed/Burstable)
- Choose appropriate storage (SSD/NVMe)
- Implement application-level caching
- Use performance testing before production
- Set alerts for CPU throttling, OOM, and I/O latency

Kubernetes Scheduling and Node Failures

VERIQTA

Scenario 9

Kubernetes Pods remain Pending despite apparently sufficient cluster resources. How do you investigate scheduler decisions, taints, affinity, topology, and resource requests?

1. Investigation Steps

1. Check Pod events and scheduler error messages.
2. Verify resource requests and limits.
3. Check node capacity and allocatable resources.
4. Investigate taints and tolerations.
5. Review node affinity, pod affinity and anti-affinity rules.
6. Check topology constraints (zone/region/hostname).
7. Verify PVC and storage class (binding issues).
8. Look for quota, LimitRange, or admission policy rejections.

2. Useful Commands

```
$ kubectl describe pod <pod>                  # view events
$ kubectl get events --sort-by=.lastTimestamp
$ kubectl get node -o wide                    # node status
$ kubectl describe node <node>                # taints, capacity
$ kubectl get pods -A -o wide                 # pod placement
$ kubectl get ns <ns> -o yaml                 # resource quotas
$ kubectl describe quota -n <ns>
$ kubectl describe limitrange -n <ns>
$ kubectl get pv,pvc -A                       # storage status
$ kubectl describe pvc <pvc>
$ kubectl get nodes -L topology.kubernetes.io/zone
$ kubectl get pods -o yaml                    # check affinity
$ kubectl get taint nodes                     # show taints
$ kubectl get pod <pod> -o yaml | grep -A 20 affinity
```

3. Common Causes

- Resource requests too high.
- No matching node due to taints.
- Missing or incorrect tolerations.
- Node/pod affinity or anti-affinity rules.
- Topology spread constraints not satisfied.
- Insufficient ephemeral storage.
- PVC not bound or storage class issues.
- Namespace resource quota exceeded.
- LimitRange restrictions.
- DaemonSet or system overhead.

4. Fixes and Prevention

- Adjust resource requests/limits.
- Add correct tolerations or remove unnecessary taints.
- Review and fix affinity/topology rules.
- Ensure storage classes and PVCs are available.
- Increase namespace quota or LimitRange.
- Use cluster autoscaler if capacity is low.
- Validate admission policies and constraints.
- Monitor scheduler events and set alerts.

Scenario 10

A Kubernetes worker node becomes NotReady during peak traffic. How do you assess workload impact, eviction behavior, recovery, and disruption risk?

1. Investigation Steps

1. Confirm node status and reason (NotReady).
2. Check node conditions and events.
3. Identify Pods running on the node.
4. Check if Pods are being evicted.
5. Assess impact on critical workloads and PDBs (Pod Disruption Budgets).
6. Investigate kubelet, container runtime and network issues.
8. Check system resources (CPU, memory, disk, network).
8. Review recent changes (deployments, upgrades, scaling, infrastructure).

2. Useful Commands

```
$ kubectl get nodes -o wide                   # node status
$ kubectl describe node <node>                # conditions
$ kubectl get events --field-selector \\
  involvedObject.name=<node>                  # node events
$ kubectl get pods -A --field-selector \
  spec.nodeName=<node>                        # pods on node
$ kubectl describe pod <pod>                  # eviction info
$ kubectl get pdb -A                          # PDB status
$ kubectl get eviction -A                     # eviction events
$ journalctl -u kubelet -n 100                # kubelet logs
$ systemctl status kubelet                    # kubelet status
$ df -h                                       # disk usage
$ free -h                                     # memory usage
$ top                                         # CPU usage
$ crictl ps                                   # container runtime
$ ip a                                        # network status
```

3. Common Causes

- Node resource exhaustion (CPU, memory, disk).
- Network issues (CNI, routing, DNS).
- Kubelet or container runtime failure.
- OS kernel issues or hardware problems.
- Disk full (including /var/lib/containerd).
- Node heartbeat loss (network partition).
- Maintenance, upgrade or reboot.
- Cloud provider node termination.
- Too many Pods or large workloads.
- Underlying infrastructure (VM, instance) issues.

4. Node Recovery Steps

- Identify and fix the root cause.
- Restart kubelet or container runtime if safe.
- Free disk space or resolve resource pressure.
- Replace unhealthy node if needed.
- Allow cluster autoscaler to provision a new node.
- Verify Pods are rescheduled and healthy.
- Check application health and latency.
- Review and improve monitoring and alerts.

5. Eviction Behavior and Impact

- Pods may be evicted if the node remains NotReady beyond the grace period (default ~5 minutes).
- PDBs can block voluntary evictions but not all failures.
- Critical workloads may lose replicas and cause outages.
- Stateful workloads can take longer to recover.
- Check eviction events and application impact.

6. Disruption Risk Mitigation

- Use multiple replicas across nodes and zones.
- Configure and test Pod Disruption Budgets (PDBs).
- Use proper resource requests and limits.
- Monitor node health, disk, memory and kubelet status.
- Use clustered autoscaling and self-healing.
- Implement infrastructure redundancy (multi-AZ).
- Regularly test failure scenarios.

7. Monitoring and Alerts

- Alert on NodeNotReady and Unreachable.
- Monitor node resource usage (CPU, memory, disk).
- Track kubelet and container runtime errors.
- Monitor eviction rate and PDB violations.
- Monitor cluster autoscaler activity.
- Set alerts for node restarts and system failures.
- Use centralized logging and metrics (e.g. Prometheus).

Kubernetes Application Outages

VERIQTA

Scenario 11

A Deployment reports available replicas, but customers receive HTTP 503 errors. How do you investigate readiness, endpoints, service routing, and ingress behavior?

1. Investigation Steps

1. Check pod health and readiness status.
2. Verify Deployment, ReplicaSet and Pod events.
3. Check if Pods are Ready and passing readiness probes.
4. Verify Endpoints and EndpointSlices have ready IPs.
5. Test Service routing inside the cluster.
6. Check Ingress, LoadBalancer and backend health.
7. Review recent changes (deployment, config, ingress, certificates).
8. Check application logs for errors.

2. Useful Commands

```
$ kubectl get deploy <name> -o wide
$ kubectl get pods -l app=<name>
$ kubectl describe pod <pod>
$ kubectl get endpoints <service>
$ kubectl get endpointslice
$ kubectl describe svc <service>
$ kubectl get ingress <name> -o yaml
$ kubectl describe ingress <name>
$ kubectl logs <pod> --previous
$ kubectl logs -l app=<name> --tail=100
$ kubectl get events --sort-by=.lastTimestamp
$ curl -v http://<service>.<ns>.svc.cluster.local
$ kubectl port-forward svc/<svc> 8080:80
```

3. Common Causes

- Readiness probe failing (pods not ready).
- Service has no ready endpoints.
- Selector mismatch (Service → Pods).
- Ingress routing to wrong service or port.
- Load balancer unhealthy targets.
- TLS or certificate issues (HTTPS 503).
- Pod running but application returning 503.
- NetworkPolicy blocking traffic.
- Recent deployment with bad configuration.
- External dependency failure (database/API).

4. Fixes and Prevention

- Fix readiness probe and startup time.
- Ensure Service selector matches labels.
- Verify endpoints have ready IPs.
- Check ingress rules, backend service and health.
- Rollback if recent deployment caused the issue.
- Validate TLS certificates and LB health.
- Check application and dependency health.
- Add monitoring and alerts for 5xx and endpoint count.

5. Traffic Flow to Check



Scenario 12

A Kubernetes application enters CrashLoopBackOff after a configuration change. What evidence determines whether to roll back, repair configuration, or investigate dependencies?

1. Investigation Steps

1. Check pod status and restart count.
2. Inspect container logs (current and previous).
3. Identify configuration changes.
4. Verify ConfigMap, Secret, and mounted files.
5. Check environment variables and runtime args.
6. Test application startup locally with the same config.
7. Check dependency connectivity (database, API, DNS).
8. Review events and describe output for exact failure reason.

2. Useful Commands

```
$ kubectl get pods -l app=<name>
$ kubectl describe pod <pod>
$ kubectl logs <pod>
$ kubectl logs <pod> --previous
$ kubectl get events --sort-by=.lastTimestamp
$ kubectl get configmap <name> -o yaml
$ kubectl get secret <name> -o yaml
$ kubectl describe configmap <name>
$ kubectl describe secret <name>
$ kubectl exec -it <pod> -- env | sort
$ kubectl exec -it <pod> -- cat /app/config.yaml
$ kubectl exec -it <pod> -- curl -v <dependency>
$ kubectl rollout history deploy/<name>
$ kubectl rollout undo deploy/<name>
```

3. Common Causes

- Invalid configuration (syntax or values).
- Missing or incorrect environment variables.
- Wrong or missing ConfigMap/Secret.
- Application cannot connect to a dependency.
- File permission issues.
- Image or version change causing incompatibility.
- Insufficient resources (CPU/memory).
- Crash during startup (init, migration, schema).
- Liveness probe killing the container.
- External service (DB, API, DNS) unreachable.

4. What to Check in Logs

- Application stack trace and error message.
- Configuration parsing errors.
- Missing files or permissions denied.
- Dependency connection failures (timeout/refused).
- OOMKilled or segmentation fault.
- Liveness probe failures.

5. Decision: Roll Back, Fix or Investigate

- Roll back: clear recent change caused the issue.
- Fix configuration: incorrect or missing config/secret.
- Investigate dependencies: external service failure.
- Check resources: restart due to OOM or CPU limits.
- Validate startup probes and commands.
- Reproduce locally with same image and config.

6. Prevention Best Practices

- Use configuration validation and schemas.
- Use separate staging environment.
- Add startup and readiness probes.
- Set resource requests and limits.
- Use immutable container images.
- Monitor restart count and CrashLoopBackOff.
- Implement automated rollback for failures.
- Add alerts for pod failures and dependency health.

Kubernetes Scaling and Resource Management

VERIQTA

Scenario 13

Traffic doubles, but the Horizontal Pod Autoscaler does not increase replicas. How do you investigate metrics, scaling policies, resource requests, and scheduling capacity?

1. Investigation Steps

1. Check current HPA status and events.
2. Verify metrics (CPU, memory or custom).
3. Review HPA configuration (min/max, metrics, targets).
4. Check resource requests and limits on the pods.
5. Verify if the metrics server is working.
6. Check if there is enough cluster capacity.
7. Look for scheduling failures or pending pods.
8. Examine recent changes (deployment, metrics, infrastructure).

2. Useful Commands

```
$ kubectl get hpa                          # HPA status
$ kubectl describe hpa <name>              # details & events
$ kubectl get pods -l app=<name>           # current replicas
$ kubectl top pods                         # resource usage
$ kubectl top nodes                        # node capacity
$ kubectl describe pod <pod>               # check requests/limits
$ kubectl get events --sort-by=.lastTimestamp
# recent events
$ kubectl get nodes                        # node resources
$ kubectl get pods -A --field-selector \
  status.phase=Pending"                    # pending pods
$ kubectl describe deploy <name>           # verify replica count
$ kubectl logs -n kube-system
  deployment/metrics-server                # metrics server logs
```

3. Common Causes

- Incorrect or missing metrics (metrics server).
- HPA misconfiguration (wrong target/metric).
- Resource requests too high (no schedulable nodes).
- Max replicas set too low.
- Custom metrics not available.
- Metrics not meeting the scaling threshold.
- Pods stuck in Pending due to taints, affinity, or insufficient resources.
- Node group/cluster autoscaler not scaling.
- Application not using CPU (wrong metric).
- Recent configuration changes.

4. Fixes and Prevention

- Ensure metrics server or custom metrics work.
- Set correct target utilization and min/max replicas.
- Right-size resource requests and limits.
- Ensure sufficient cluster capacity or enable cluster autoscaler.
- Check and fix taints/affinity that block scheduling.
- Use appropriate metrics (CPU, memory, custom).
- Monitor HPA events and set alerts.

5. HPA Flow Overview



Scenario 14

A Kubernetes cluster scales successfully but application latency worsens. How do you investigate downstream bottlenecks, connection limits, and shared dependencies?

1. Investigation Steps

1. Confirm that scaling actually happened.
2. Check application and infrastructure latency.
3. Identify which component is slower (app, DB, cache, external API).
4. Check connection limits and pool usage.
5. Review downstream dependencies.
6. Examine resource usage on new pods and nodes.
7. Look for network, DNS, or load balancer issues.
8. Compare performance metrics before and after scaling.

2. Useful Commands

```
$ kubectl get pods -l app=<name>   # verify replica count
$ kubectl top pods                 # resource usage
$ kubectl logs <pod> --tail=100    # application logs
$ kubectl exec -it <pod> -- curl -w "time: %{time_total}\n"
  https://api.example.com          # test downstream latency
$ kubectl exec -it <pod> -- ss -antp    # check connections
$ kubectl exec -it <pod> -- netstat -anp  # network connections
$ kubectl describe svc <service>   # service endpoints
$ kubectl get endpoints <service>  # endpoint count
$ kubectl top nodes                # node resource usage
$ curl -I https://service.example.com  # check external latency
$ dig service.example.com          # DNS resolution
```

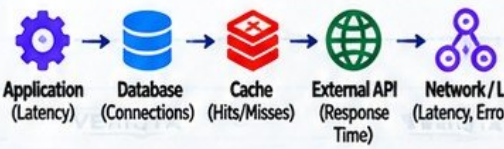
3. Common Causes

- Downstream database or API bottleneck.
- Connection pool limits (DB, cache, HTTP).
- Insufficient backend capacity.
- Load balancer saturation or misconfiguration.
- High latency in external dependencies.
- DNS resolution delays.
- Increased contention for shared resources.
- Inefficient application code or thread pools.
- Network latency between services or regions.
- More replicas increasing load on a shared dependency.

4. What to Check

- Application latency and error rate (APM, logs).
- Database metrics (connections, slow queries).
- Cache hit/miss rate and latency.
- External API response times.
- Load balancer and ingress metrics.
- Network metrics (latency, packet loss).
- Pod and node resource usage after scaling.
- Connection pool and thread pool configurations.

5. Investigation Flow



6. Fixes and Prevention

- Increase downstream capacity (DB, cache, API).
- Tune connection pools and limits.
- Optimize queries and application code.
- Use caching and reduce external calls.
- Set proper load balancer timeouts and health checks.
- Monitor end-to-end latency with distributed tracing.
- Implement rate limiting and circuit breakers.
- Capacity planning and load testing before scaling.

CI/CD Pipeline Reliability

VERIQTA

Scenario 15

A production deployment succeeds in the CI/CD pipeline, but the application becomes unavailable. How would you redesign release verification and deployment safety?

1. Investigation Steps

1. Check deployment logs and pipeline output.
2. Verify what was actually deployed (image tag, commit, config).
3. Check application, Kubernetes and infrastructure health.
4. Compare prod environment with staging (config, secrets, dependencies).
5. Review recent changes in code, config, database or infrastructure.
6. Check rollback options and previous working version.
7. Analyze monitoring, alerts and health checks.
8. Identify gaps in release verification.

2. Useful Commands

```
$ kubectl get deployments                    # check status
$ kubectl get pods -n <namespace>            # pod status
$ kubectl get events -n <namespace>          # events
$ kubectl rollout history deployment/<name>
$ kubectl describe deployment/<name>
$ kubectl logs -n <namespace> <pod>          # application logs
$ kubectl get rs -n <namespace>              # replica sets
$ kubectl get svc,ep -n <namespace>          # service endpoints
$ kubectl get ingress -n <namespace>         # ingress status
$ kubectl rollout undo deployment/<name>     # rollback
$ kubectl get configmap,secret -n <ns>       # config/secrets
$ docker inspect <image>                     # image details
$ curl -v http://<service>                   # test endpoint
$ kubectl get nodes                          # node health
```

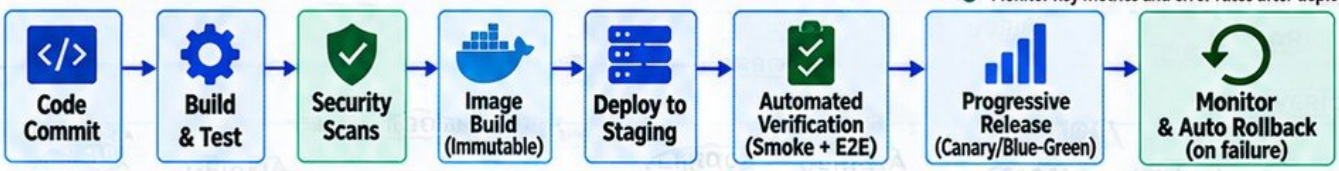
3. Common Causes

- Incorrect or missing configuration.
- Wrong image tag or build artifact.
- Database migration issues.
- Missing or wrong environment variables.
- Service endpoints not updated.
- Ingress or load balancer misconfiguration.
- Health checks passing but app not fully ready.
- Dependency or external service failure.
- Feature flags not set correctly.
- Pipeline does not test production-like conditions.

4. Fixes and Prevention

- Add automated smoke and integration tests.
- Use staging environment that mirrors production.
- Implement blue/green or canary deployments.
- Add health checks with real user journeys.
- Validate database migrations before rollout.
- Use feature flags for safe rollout.
- Implement automated rollback on failure.
- Monitor key metrics and error rates after deployment.

5. Improved Deployment Safety Flow



Scenario 16

A critical release must reach production quickly, but security scans and approval gates are failing. How do you balance delivery urgency, risk, and governance?

1. Investigation Steps

1. Understand what is blocking the release (security scan, policy, approval).
2. Review the severity of the findings (critical, high, medium, low).
3. Assess the actual risk and business impact.
4. Check if findings are false positives.
5. Identify required approvals and who can authorize exceptions.
6. Evaluate mitigation options for high-risk issues.
7. Document the decision, risk and approval.
8. Plan post-deployment fixes and timelines.

2. Useful Commands

```
# View security scan results (example tools)
trivy image <image>                          # vulnerability scan
grype <image>                                # vulnerability scan
snyk test                                    # dependency scan
# Check pipeline logs
kubectl describe pipeline <name>             # if using ArgoCD/CI
kubectl get argo rollouts                    # rollout status
kubectl get events -n <namespace>            # cluster events
# Check policy and approvals (example)
opa eval --data policies/ --input input.json
# View image and dependencies
docker sbom <image>                          # software bill of materials
docker inspect <image>                       # image details
```

3. Key Considerations

- Severity and exploitability of the findings.
- Whether the issue is a false positive.
- Business impact of the delay.
- Compensating controls (WAF, network rules).
- Can the risk be mitigated temporarily?
- Who can approve an exception?
- Regulatory or compliance requirements.
- Plan and timeline to fix the issues.
- Monitor closely after release.
- Communicate risk and decision to stakeholders.

4. Options for Handling the Release

- Fix the issues and re-run scans.
- Request a time-bound exception with justification.
- Deploy with compensating controls (e.g., WAF, network rules).
- Release only to a limited audience (canary).
- Disable or isolate the affected component.
- Use feature flags and enable gradually.
- If risk is unacceptable, delay the release.

5. Decision Framework

- Assess risk (severity × likelihood × impact).
- Involve security, product and engineering stakeholders.
- Choose the safest option that meets business needs.
- Document the decision and approval.
- Set a clear timeline to fix the issues.
- Monitor closely and be ready to roll back.
- Learn and improve the process.

6. Long-Term Improvements

- Shift security left in the development process.
- Automate security scanning in CI/CD.
- Use clear risk-based approval policies.
- Maintain a fast-track process for urgent releases.
- Reduce false positives through better tooling.
- Improve dependency and image management.
- Regularly review and update security policies.

GitOps and Deployment Recovery

VERIQTA

Scenario 17

Argo CD repeatedly reverses emergency production changes. How do you investigate reconciliation, Git state, drift, and emergency change procedures?

1. Investigation Steps

1. Check Argo CD application status and sync history.
2. Verify the current live state vs Git state (drift).
3. Review recent commits and who made the changes.
4. Check Argo CD sync policy, automated sync and prune settings.
5. Examine Kubernetes events and resource changes.
6. Verify whether emergency changes were made directly in the cluster.
7. Check RBAC and access controls.
8. Identify the correct procedure for emergency changes.

2. Useful Commands

```
$ argocd app list                          # list applications
$ argocd app get <name>                    # app details
$ argocd app history <name>                # sync history
$ argocd app diff <name>                   # compare live vs Git
$ argocd app manifests <name>              # show desired state
$ argocd app logs <name>                   # controller logs
$ argocd app sync <name>                   # manual sync
$ kubectl get all -n <namespace>           # current resources
$ kubectl describe <resource>              # check events
$ kubectl get events -n <ns>               # recent events
$ kubectl get configmap -n <ns>            # check config
$ kubectl get secret -n <ns>               # check secrets
$ kubectl auth can-i <verb> <resource>     # RBAC check
```

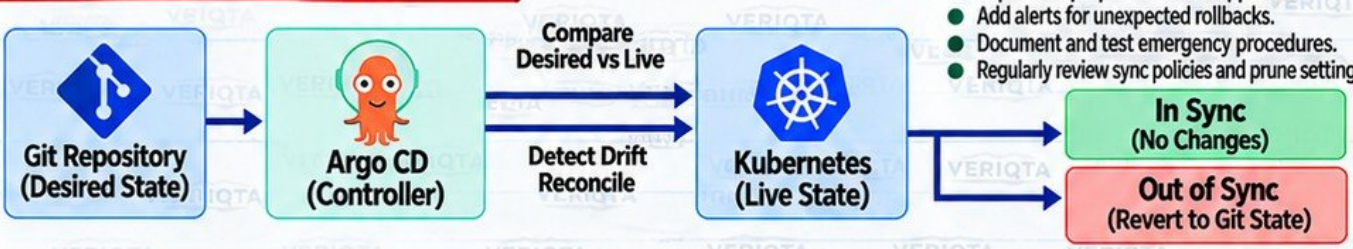
3. Common Causes

- Automated sync reverting manual changes.
- Git state is older than the live change.
- Drift detection reconciling to Git.
- Misconfigured sync policy (auto-sync, prune).
- Changes made directly in the cluster instead of Git.
- Missing or delayed Git commits.
- Multiple environments using the same repo.
- RBAC or process gaps for emergency changes.
- Webhook or CI updates overwriting changes.
- Out-of-band changes without proper approval.

4. Fixes and Prevention

- Make all changes through Git and pull requests.
- Use a documented emergency change process (temporary override or pause sync).
- Configure sync windows for critical periods.
- Use selective sync or app ignore rules when needed.
- Implement proper RBAC and approvals.
- Add alerts for unexpected rollbacks.
- Document and test emergency procedures.
- Regularly review sync policies and prune settings.

5. Argo CD Reconciliation Flow



Scenario 18

A Kubernetes rolling deployment causes intermittent customer failures despite passing health checks. How would you redesign probes, traffic draining, and rollout controls?

1. Investigation Steps

1. Check the deployment status and rollout history.
2. Review pod readiness and liveness probe configuration.
3. Examine Kubernetes events and pod logs.
4. Check service endpoints and traffic routing during rollout.
5. Analyze termination grace period and preStop hooks.
6. Verify load balancer and ingress health check behavior.
7. Look for connection draining and session handling issues.
8. Review application startup time and dependency readiness.

2. Useful Commands

```
$ kubectl get deploy <name>                # deployment status
$ kubectl rollout status deploy/<name>     # rollout progress
$ kubectl rollout history deploy/<name>    # revision history
$ kubectl describe deploy <name>           # events and config
$ kubectl get pods -l app=<name>           # pod status
$ kubectl describe pod <pod>               # pod details
$ kubectl logs <pod>                       # application logs
$ kubectl get endpoints <service>          # service endpoints
$ kubectl describe svc <service>           # service details
$ kubectl get events -n <ns>               # recent events
$ kubectl get ingress <name>               # ingress config
$ kubectl describe ingress <name>          # ingress details
$ kubectl get pod -o wide                  # node placement
```

3. Common Causes

- Readiness probe passing before the app is fully ready.
- Liveness probe killing healthy pods.
- Insufficient preStop or termination grace period.
- Load balancer sending traffic to terminating pods.
- Session persistence or connection draining issues.
- Slow application startup or dependency initialization.
- Incorrect probe endpoints or thresholds.
- Resource limits causing slow response.
- Ingress or service misconfiguration.
- Database or external dependency latency.

4. Probe and Rollout Improvements

- Use realistic readiness and liveness probes.
- Set appropriate initialDelay, period and failure thresholds.
- Implement preStop hooks and graceful shutdown.
- Increase terminationGracePeriodSeconds.
- Use rolling update with maxUnavailable and maxSurge.
- Test probes under real load.

5. Traffic Draining and Rollout Controls

- Enable connection draining on load balancer/ingress.
- Use Pod Disruption Budgets (PDBs).
- Implement canary or blue/green deployment strategy.
- Tune rollout settings (maxSurge, maxUnavailable).
- Monitor error rates, latency and success rate during rollout.
- Use automated rollback on failure.

6. Monitoring and Validation

- Monitor readiness probe success rate.
- Track HTTP 5xx errors and latency.
- Check service endpoint health during rollout.
- Validate load balancer target health.
- Use distributed tracing and logs.
- Conduct a staged rollout with monitoring.

Terraform and Infrastructure State

VERIQTA

Scenario 19

A Terraform plan proposes replacing a production database. What would you investigate before approving or rejecting the change?

1. Investigation Steps

1. Review the plan output in detail and identify the exact change.
2. Check why Terraform wants to replace the database.
3. Verify configuration changes (variable, resource, provider).
4. Check for drift between state and actual infrastructure.
5. Review dependencies that will also be affected.
6. Assess data protection, backup and replication.
7. Check maintenance window, downtime impact and rollback plan.
8. Validate that the change is intentional and approved.

2. Useful Commands

```
$ terraform plan                          # review proposed changes
$ terraform plan -detailed-exitcode       # CI/CD check
$ terraform show -no-color plan.out       # detailed output
$ terraform state show <resource>         # current state
$ terraform state list                    # list all resources
$ terraform state pull                    # view full state
$ terraform refresh                       # sync with real infra
$ terraform providers                     # check provider versions
$ terraform workspace show                # current workspace
$ terraform graph | dot -Tpng > graph.png # visualize deps
$ terraform plan -target=<resource>       # focused plan
$ terraform console                       # inspect variables
$ terraform validate                      # check configuration
```

3. Common Causes

- Immutable field change (e.g. engine, storage).
- Configuration change that forces replacement.
- Resource drift (state does not match real infra).
- Provider or module version changes.
- Accidental change to resource name or key.
- Lifecycle rules not set (create_before_destroy).
- Missing or incorrect variables.
- Environment or workspace mix-up.
- Manual changes in the cloud console.
- Module update with breaking changes.

4. Fixes and Prevention

- Understand the reason for replacement.
- Use lifecycle rules (create_before_destroy, prevent_destroy).
- Validate configuration and variables.
- Keep state in remote backend with locking.
- Review changes with a peer before apply.
- Use Terraform plan in CI/CD with approvals.
- Maintain regular backups and snapshots.
- Test changes in a staging environment.
- Document and review high-impact changes.

5. Terraform Plan Example (Replacement)

```
# terraform plan
- resource "aws_db_instance" "prod_db" {
    - identifier = "prod-db"
    - engine_version = "13.11"
  -/+ instance_class = "db.r5.large" -> "db.r6g.large"
      (forces replacement)
    ± ... (other changes)
}
```

Why Replacement Happens

- Changing instance class (not supported in-place)
- Changing engine version
- Changing storage type or size
- Changing availability settings
- Changing subnet group or parameter group
- Immutable field modification
- Resource name change

Before Approving

- ✓ Confirm the change is intentional.
- ✓ Validate impact and downtime.
- ✓ Check backups and snapshots.
- ✓ Review dependencies.
- ✓ Ensure rollback plan is ready.
- ✓ Get proper approvals.
- ✓ Test in a non-production environment.
- ✓ Communicate with stakeholders.

Scenario 20

Two engineers apply Terraform changes simultaneously, resulting in conflicting infrastructure state. How do you recover safely and prevent recurrence?

1. Investigation Steps

1. Identify what changes were applied by each engineer.
2. Check the current state and recent state history.
3. Compare the state file with real infrastructure.
4. Look for conflicting changes in the Terraform logs and cloud provider.
5. Assess the impact on resources.
6. Determine if any manual changes were made.
7. Decide on the correct desired state.
8. Plan a safe recovery and prevent future conflicts.

2. Useful Commands

```
$ terraform plan                          # check current changes
$ terraform show                          # view current state
$ terraform state pull                    # download state file
$ terraform state list                    # list resources
$ terraform state show <resource>         # inspect resource
$ terraform refresh                       # sync with real infra
$ terraform import <type> <name> <id>     # re-import if needed
$ terraform state rm <resource>           # remove bad resource
$ terraform apply -refresh-only           # fix drift
$ terraform force-unlock <lock-id>        # clear stuck lock
$ terraform workspace list                # check workspaces
$ terraform plan -out=plan.out            # create recovery plan
```

3. Common Causes

- No remote state locking (e.g. local state).
- Two engineers applying at the same time.
- State file manually edited or out of sync.
- Using different workspaces or backends.
- Network or CI/CD concurrency issues.
- Partial apply or interrupted run.
- Manual changes in the cloud console.
- Lack of change coordination and approvals.
- Missing state lock configuration (S3 + DynamoDB).
- Automation pipelines running simultaneously.

4. Recovery Steps

- Stop further changes immediately.
- Take a backup of the current state file.
- Identify the correct state (from logs or cloud provider).
- Reconcile state with real infrastructure (refresh/import).
- Manually fix any drift or missing resources.
- Apply a controlled plan to reach the desired state.
- Validate the environment and test critical resources.
- Resume normal operations after verification.

5. Preventing Recurrence

- Use remote state with locking (e.g. S3 + DynamoDB).
- Enforce a single pipeline or approved workflow.
- Use workspace strategy for environments.
- Implement pull request reviews for Terraform changes.
- Add automated plan checks in CI/CD.
- Use clear change windows and team communication.
- Enable notifications for state changes.
- Provide training and documentation for the team.

6. Best Practices

- Always use remote state with locking.
- Never share local state files.
- Use peer review and approvals.
- Run terraform plan before apply.
- Automate with CI/CD and governance.
- Keep audit logs of changes.
- Monitor infrastructure for unexpected changes.
- Document recovery procedures.

AWS Architecture and High Availability

VERIQTA

Scenario 21

An application becomes unavailable when one AWS Availability Zone experiences a failure. How would you identify hidden single points of failure?

1. Investigation Steps

1. Confirm which AZ is affected and what is unhealthy.
2. Check the architecture and multi-AZ configuration.
3. Verify load balancer, target groups, and cross-AZ settings.
4. Check database, cache, and storage configurations.
5. Review DNS, health checks, and failover behavior.
6. Identify any components running in a single AZ.
7. Check recent changes and deployments.
8. Validate monitoring, alarms and failure events.

2. Useful Commands (AWS CLI)

```
$ aws ec2 describe-availability-zones   # check AZ status
$ aws elbv2 describe-load-balancers     # check LB config
$ aws elbv2 describe-target-groups      # check targets
$ aws elbv2 describe-target-health \
  --target-group-arn <arn>              # target health
$ aws rds describe-db-instances         # DB multi-AZ
$ aws elasticache describe-cache-clusters   # cache AZs
$ aws autoscaling describe-auto-scaling-groups  # ASG AZs
$ aws ec2 describe-instances            # instance AZs
$ aws route53 get-health-checks         # DNS health checks
$ aws cloudwatch describe-alarms        # recent alarms
$ aws cloudtrail lookup-events \
  -lookup-attributes AttributeKey=EventName,\
  AttributeValue=TerminateInstances     # recent events
```

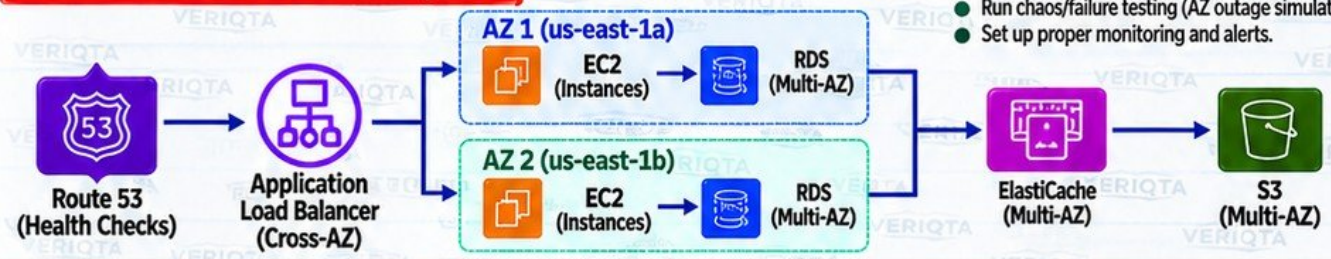
3. Common Causes

- Critical component deployed in a single AZ.
- Load balancer not configured for cross-AZ.
- RDS, ElastiCache or stateful service not multi-AZ.
- AZ-specific subnets or routing issues.
- DNS configuration pointing to unhealthy targets.
- Auto Scaling Group limited to one AZ.
- Shared services (NAT, bastion, middleware) in a single AZ.
- Health checks misconfigured or too lenient.
- Hidden dependencies (e.g., third-party, on-prem).
- Recent deployment or configuration change.

4. Fixes and Prevention

- Deploy all critical components across multiple AZs.
- Enable cross-AZ load balancing.
- Use multi-AZ for RDS, ElastiCache and other stateful services.
- Remove single-AZ dependencies (NAT, DNS, etc.).
- Validate health checks and failover regularly.
- Implement Infrastructure as Code with multi-AZ design.
- Run chaos/failure testing (AZ outage simulations).
- Set up proper monitoring and alerts.

5. Multi-AZ Architecture Example



Scenario 22

An AWS architecture is highly available but costs significantly more than expected. How would you reduce costs without compromising agreed reliability targets?

1. Investigation Steps

1. Identify the main cost drivers by service.
2. Review architecture, redundancy and utilization.
3. Check if all resources are necessary for the reliability target.
4. Analyze idle, underutilized or oversized resources.
5. Review data transfer, storage and backup costs.
6. Validate if cheaper alternatives or right-sized options exist.
7. Compare actual usage with the agreed SLA and reliability requirements.
8. Identify optimization opportunities without reducing resilience.

2. Useful Commands (AWS CLI)

```
$ aws ce get-cost-and-usage \
  --time-period Start=2026-09-01,End=2026-10-01 \
  --granularity DAILY --metrics BlendedCost \
  --group-by Type=DIMENSION,Key=SERVICE  # cost by service
$ aws ce get-cost-and-usage \
  --group-by Type=DIMENSION,Key=REGION   # cost by region
$ aws ce get-cost-and-usage \
  --group-by Type=DIMENSION,Key=USAGE_TYPE   # by usage type
$ aws rds describe-db-instances         # DB instance details
$ aws ec2 describe-instances            # check instance sizes
$ aws autoscaling describe-auto-scaling-groups  # ASG config
$ aws s3 ls s3://<bucket> --summarize   # storage usage
$ aws cloudwatch get-metric-statistics  # utilization metrics
$ aws ce get-reservation-coverage       # RI/SP coverage
$ aws ce get-savings-plans-utilization  # Savings Plans
```

3. Common Cost Drivers

- Over-provisioned instances and databases.
- Unnecessary multi-AZ where not required.
- Underutilized or idle resources.
- High data transfer between regions or AZs.
- Large storage and backup retention.
- NAT Gateway and inter-AZ traffic charges.
- Unused load balancers, EBS volumes, snapshots.
- Expensive instance types instead of right-sizing.
- Lack of Reserved Instances or Savings Plans.
- Inefficient architecture or duplicated resources.

4. Cost Optimization Strategies

- Right-size instances, databases and storage.
- Use Auto Scaling with appropriate min/max.
- Use Spot Instances for non-critical workloads.
- Leverage Reserved Instances or Savings Plans.
- Optimize storage (lifecycle policies, compression).
- Reduce unnecessary data transfer and inter-AZ traffic.
- Remove unused resources (instances, volumes, load balancers).
- Use cost-effective services (S3 tiers, serverless, managed services).

5. Maintain Reliability Targets

- Keep multi-AZ for critical components (as required).
- Use Auto Scaling and health checks.
- Maintain backups and disaster recovery.
- Set clear SLA/SLI and monitor them.
- Test failover and recovery regularly.
- Choose cost optimizations that do not reduce resilience.
- Implement changes gradually and measure impact.
- Review architecture with stakeholders.

6. Monitoring and Governance

- Set up cost budgets and alerts.
- Use AWS Cost Anomaly Detection.
- Tag all resources for cost allocation.
- Regularly review usage and remove waste.
- Enforce policies with AWS Config and IAM.
- Use Infrastructure as Code for consistency.
- Review costs vs reliability targets monthly.
- Document and communicate cost decisions.

Cloud Networking and IAM

VERIQTA

Scenario 23

Private EC2 instances cannot reach a required external service after a VPC networking change. How do you investigate routes, NAT, security groups, and network ACLs?

1. Investigation Steps

1. Verify the exact error and destination (service, domain, IP, port).
2. Check the subnet and route table.
3. Confirm NAT Gateway/Instance availability and routes.
4. Review security groups (outbound and inbound rules).
5. Check network ACLs (subnet level).
6. Test DNS resolution and connectivity from the instance.
7. Compare with a working instance or previous configuration.
8. Identify what changed in the VPC networking.

2. Useful Commands (AWS CLI & Linux)

```
# From EC2 instance
$ ping <domain>                  # test DNS resolution
$ curl -v https://<domain>       # test connectivity
$ nslookup <domain>              # DNS lookup
$ traceroute <domain>            # trace network path
$ telnet <domain> 443            # test specific port
# AWS CLI (from local machine)
$ aws ec2 describe-route-tables \
    --filters Name=vpc-id,Values=<vpc-id>
$ aws ec2 describe-nat-gateways \
    --filter Name=vpc-id,Values=<vpc-id>
$ aws ec2 describe-security-groups \
    --group-ids <sg-id>
$ aws ec2 describe-network-acls \
    --filters Name=vpc-id,Values=<vpc-id>
```

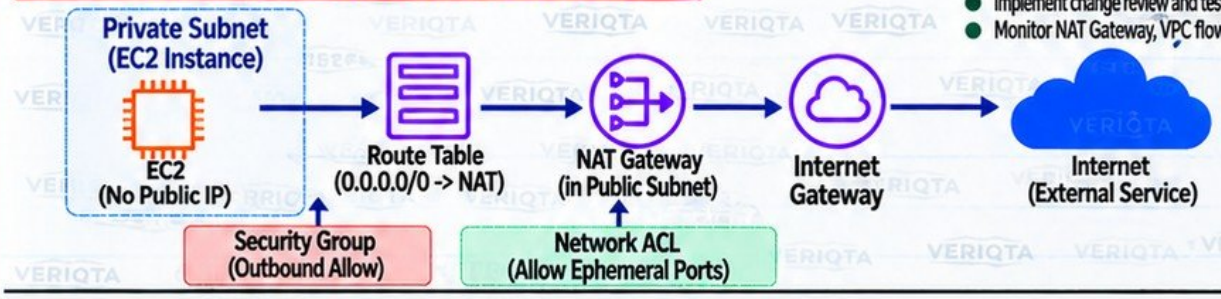
3. Common Causes

- Missing or incorrect route to internet.
- NAT Gateway not available or not in the route table.
- Security group blocking outbound traffic.
- Network ACL denying outbound or return traffic.
- DNS resolution failure (Route 53, VPC DNS).
- Using private endpoint instead of internet.
- Incorrect subnet association with route table.
- NAT Gateway in a different AZ or misconfigured.
- VPC endpoints or firewall blocking the service.
- Recent VPC changes (routes, ACLs, subnets).

4. Fixes and Prevention

- Add correct route to NAT Gateway (0.0.0.0/0).
- Ensure NAT Gateway is healthy and in the same AZ.
- Allow required outbound traffic in security group.
- Check and update network ACL rules (ephemeral ports).
- Verify DNS settings and use Route 53 Resolver if needed.
- Use VPC endpoints for AWS services where possible.
- Implement change review and testing before VPC changes.
- Monitor NAT Gateway, VPC flow logs and set alerts.

5. VPC Networking Flow (Private to Internet)



Scenario 24

A production workload suddenly loses access to an AWS service. How would you investigate IAM policies, assumed roles, resource policies, and organizational restrictions?

1. Investigation Steps

1. Identify the exact error message (e.g., AccessDenied, UnauthorizedOperation).
2. Determine which AWS service and action are failing.
3. Check the IAM role or user used by the workload.
4. Review attached policies, permission boundaries and session policies.
5. Check resource-based policies (e.g., S3, KMS).
6. Verify if the role is assumed correctly and not expired.
7. Check AWS Organizations and Service Control Policies (SCPs).
8. Review recent changes to IAM, policies or organizational settings.

2. Useful Commands (AWS CLI)

```
# Check caller identity
$ aws sts get-caller-identity      # who am I
# Check IAM role details
$ aws iam get-role --role-name <name>
$ aws iam list-attached-role-policies\
    --role-name <name>
$ aws iam list-role-policies --role-name <name>
$ aws iam get-role-policy \
    --role-name <name> --policy-name <policy>
# Simulate permissions
$ aws iam simulate-principal-policy \
  --policy-source-arn arn:aws:iam::<id>:role/<name>\
  --action-names <service:Action> \
  --resource-arns <resource-arn>
# Check resource policies (example S3)
$ aws s3api get-bucket-policy --bucket <name>
# Check organizations SCPs
$ aws organizations describe-effective-policy \
  --policy-type SERVICE_CONTROL_POLICY \
  --target-id <account-id>
```

3. Common Causes

- IAM policy missing required permissions.
- Explicit deny in IAM policy or SCP.
- Resource policy (e.g., S3, KMS) blocking access.
- Assumed role not configured or session expired.
- Permission boundary restricting actions.
- MFA or conditional access not met.
- Wrong region or account.
- Service control policy (SCP) denying the action.
- Recent IAM or organizational policy changes.
- Cross-account trust relationship issues.

4. Recovery Steps

- Identify the exact deny and reason.
- Restore correct permissions (least privilege).
- Fix trust policy or role assumption issues.
- Update resource policy if required.
- Remove or adjust restrictive SCPs or permission boundaries.
- Test access with the same role and action.
- Monitor for similar failures and set alerts.

5. Prevention and Best Practices

- Use least privilege IAM policies.
- Test permission changes in non-production.
- Use IAM Access Analyzer and policy validation.
- Document and review SCPs and permission boundaries.
- Use infrastructure as code for IAM (e.g., Terraform).
- Implement monitoring and alerts for AccessDenied errors.
- Regularly audit unused permissions and roles.

6. Monitoring and Governance

- Enable CloudTrail for all accounts.
- Monitor AccessDenied and authorization errors.
- Use AWS Config and IAM Access Analyzer.
- Review SCPs and organizational changes.
- Set alerts for unusual permission changes.
- Regularly review and rotate roles and credentials.
- Implement change management for IAM updates.

Monitoring and Observability

VERIQTA

Scenario 25

Prometheus dashboards show healthy infrastructure while customers report widespread failures. What observability gaps would you investigate?

1. Investigation Steps

1. Verify if the issue is user-facing or internal only.
2. Check what metrics are actually being collected and what is missing.
3. Compare infrastructure metrics vs application metrics.
4. Check logs, traces and error rates across services.
5. Validate monitoring coverage across all environments and regions.
6. Look for blind spots (e.g., external dependencies, third-party services).
7. Verify alerting rules and thresholds.
8. Reproduce the issue from a user perspective (synthetic or real user).

2. Useful Commands (Prometheus, Grafana, Kubernetes)

```
$ kubectl get pods -n monitoring              # check monitoring stack
$ kubectl get svc -n monitoring               # check service endpoints
$ curl -s http://prometheus:9090/-/healthy    # Prometheus health
$ curl -s http://prometheus:9090/api/v1/rules  # check alert rules
$ curl -s "http://prometheus:9090/api/v1/query?
  query=up"                                   # check target status
$ kubectl logs -l app=grafana -n monitoring   # Grafana logs
$ kubectl port-forward svc/prometheus 9090:9090  # access UI
$ kubectl port-forward svc/grafana 3000:3000     # access UI
# Example PromQL queries
$ rate(http_requests_total[5m])               # request rate
$ histogram_quantile(0.95,rate(http_request_duration_
  seconds_bucket[5m]))                        # 95th percentile latency
$ sum(rate(http_requests_total{status=~"5.."}[5m]))  # 5xx errors
$ up                                          # target health
```

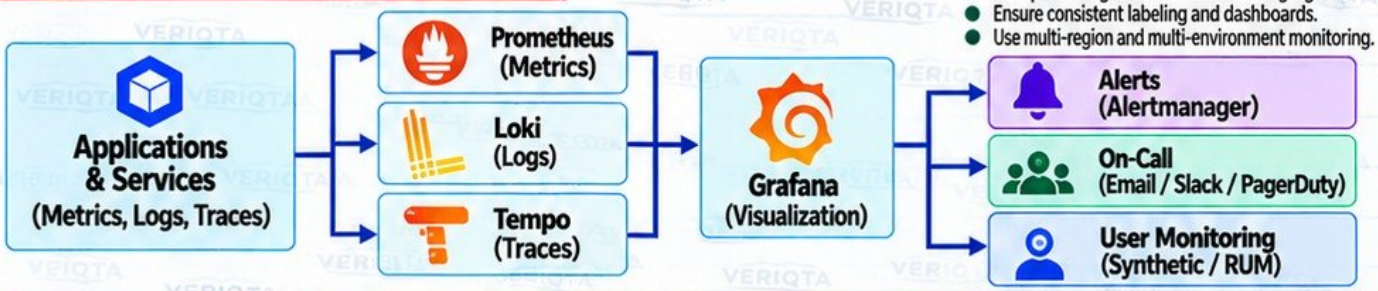
3. Common Observability Gaps

- Missing application-level metrics (e.g., business transactions).
- Metrics only for infrastructure, not user experience.
- Incorrect or missing labels and dimensions.
- No end-to-end tracing across services.
- Lack of visibility into external dependencies (3rd-party APIs, databases, payment systems).
- Synthetic monitoring not configured.
- Dashboards showing averages instead of percentiles.
- Latency, error rate or saturation not tracked.
- Monitoring not deployed in all environments/regions.
- Logs and traces not correlated with metrics.
- Alerting only on infrastructure health.

4. Fixes and Prevention

- Add application and business metrics (SLIs).
- Implement distributed tracing (e.g., OpenTelemetry).
- Monitor external dependencies.
- Use real user monitoring (RUM) and synthetic checks.
- Track RED/USE metrics (Rate, Errors, Duration; Utilization, Saturation, Errors).
- Set up meaningful alerts on user-facing signals.
- Ensure consistent labeling and dashboards.
- Use multi-region and multi-environment monitoring.

5. Observability Architecture (Example)



Scenario 26

An organization receives thousands of alerts daily, but critical incidents still go undetected. How would you redesign alerting around actionable signals and SLOs?

1. Investigation Steps

1. Understand current alert sources and volumes.
2. Review recent incidents and whether alerts fired.
3. Identify noisy or low-value alerts.
4. Check alert thresholds and conditions.
5. Analyze alert routing, escalation and on-call workflow.
6. Verify whether alerts are aligned with SLOs and business impact.
7. Assess missing alerting for critical user-facing services.
8. Review alert fatigue and team feedback.

2. Useful Commands (Prometheus & Alerting)

```
$ curl -s http://alertmanager:9093/api/v2/alerts  # list alerts
$ curl -s http://prometheus:9090/api/v1/alerts    # prometheus alerts
$ curl -s http://prometheus:9090/api/v1/rules     # alerting rules
$ kubectl get configmap alertmanager -n monitoring # alert config
$ kubectl logs -l app=alertmanager -n monitoring  # alertmanager logs
$ kubectl get prometheusrule -n monitoring        # k8s alert rules
# Example: check firing alerts
$ curl -s 'http://prometheus:9090/api/v1/alerts'| jq'.data.alerts[] |
  {name: .labels.alertname, state: .state, value: .value}'
# Example: check SLO-related metrics
$ prometql> rate(http_requests_total[5m])
$ prometql> histogram_quantile(0.99, rate(http_request_
  duration_seconds_bucket[5m]))
$ prometql> sum(rate(http_requests_total{status=~"5.."}[5m]))
```

3. Common Root Causes

- Too many low-severity or non-actionable alerts.
- Alerting on symptoms instead of root causes.
- No clear severity levels or prioritization.
- Alerts not aligned with SLOs or user impact.
- Missing alerts for critical business services.
- Incorrect thresholds (too sensitive or too lenient).
- Lack of alert deduplication and grouping.
- Unclear ownership and escalation paths.
- Alert fatigue causing real incidents to be missed.
- No runbooks or context in alerts.
- Tooling misconfiguration (e.g., duplicate alerts).
- Cross-team communication gaps.

4. Better Alerting Strategy

- Define SLOs/SLIs and alert on error budgets.
- Focus on user-facing and business-impacting signals.
- Use multi-window, multi-burn-rate alerts.
- Reduce noise with proper thresholds and filtering.
- Group and deduplicate related alerts.
- Use clear severity levels (P1, P2, P3).
- Include runbooks and links in alerts.
- Regularly review and tune alert rules.

5. Example SLO-Based Alerting

- Track key SLIs: availability, latency, error rate.
- Use error budget burn rate alerting (e.g., 2% in 1h, 5% in 6h).
- Alert on sustained user impact, not single spikes.
- Monitor both infrastructure and application SLIs.
- Set different thresholds for warning and critical.
- Use dashboards to visualize SLOs and error budgets.
- Suspend or mute alerts during planned maintenance.
- Continuously improve based on post-incident reviews.

6. Monitoring and Governance

- Assign alert ownership to service teams.
- Use on-call rotation and escalation policies.
- Integrate with incident management tools.
- Track alert volume and mean time to detect (MTTD).
- Conduct regular alert reviews and clean up.
- Ensure alerts cover all critical services and dependencies.
- Document alerting standards and practices.
- Train teams on using and responding to alerts.

Site Reliability and Incident Management

VERIQTA

Scenario 27

A major production incident begins immediately after deployment. Do you roll back, scale up, disable a feature, or investigate a shared dependency first?

1. Investigation Steps

1. Confirm the exact symptoms, scope and timeline.
2. Check recent deployments and changes.
3. Review application, infrastructure and dependency metrics.
4. Compare current state with pre-deployment baseline.
5. Identify affected services and user impact.
6. Check error rates, latency, saturation and logs.
7. Investigate shared dependencies (databases, caches, external APIs).
8. Decide the safest and fastest mitigation based on evidence.

2. Useful Commands (Kubernetes & AWS)

```
# Check recent deployments
$ kubectl get deploy -n <ns>              # list deployments
$ kubectl rollout history deploy/<name>   # rollout history
$ kubectl describe deploy/<name>          # deployment details
# Check pods and status
$ kubectl get pods -n <ns> -o wide        # pod status
$ kubectl describe pod <pod>              # pod events
$ kubectl logs <pod> --previous           # previous container logs
# Check metrics (if available)
$ kubectl top pods -n <ns>                # pod resource usage
$ kubectl top nodes                       # node resource usage
# Check recent changes in AWS (example)
aws cloudtrail lookup-events \
  --lookup-attributes AttributeKey=EventName,\
  AttributeValue=UpdateService,AttributeValue=Deploy
```

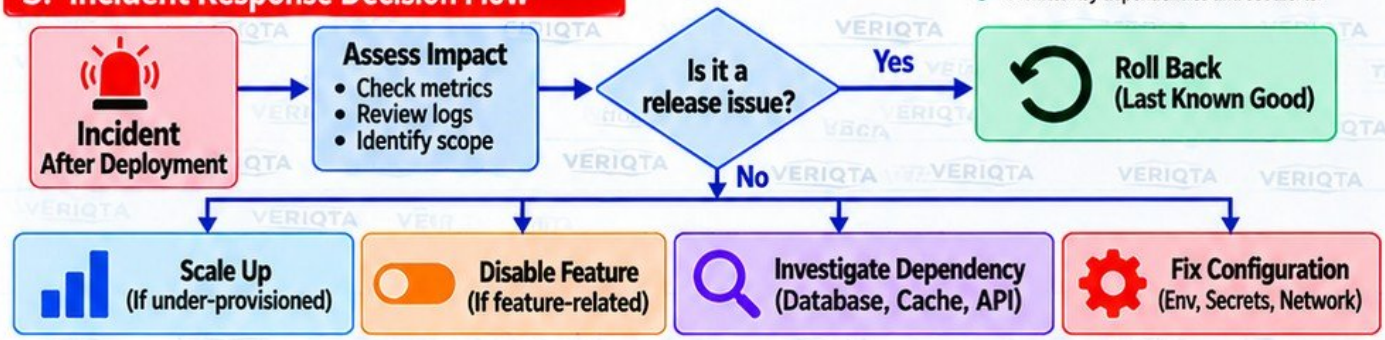
3. Common Causes

- Bad release or configuration change.
- Incompatible database or schema change.
- Downstream dependency failure.
- Resource exhaustion (CPU, memory, connections).
- Feature flag or new feature causing errors.
- Traffic spike after deployment.
- Misconfigured environment variables or secrets.
- Network or DNS issues to external services.
- Cache invalidation or data inconsistency.
- Autoscaling not keeping up with load.

4. Fixes and Prevention

- Roll back to last known good version (if release issue).
- Scale up (short-term) if under-provisioned.
- Disable the feature using feature flags.
- Fix configuration, database or dependency issues.
- Add pre-deployment and post-deployment checks.
- Use canary or blue/green deployments.
- Implement automated rollback on high error rates.
- Monitor key dependencies and set alerts.

5. Incident Response Decision Flow



Scenario 28

A service repeatedly violates its latency SLO despite high infrastructure availability. How would you use error budgets and performance evidence to prioritize corrective work?

1. Investigation Steps

1. Confirm the SLO and current error budget consumption.
2. Analyze latency metrics (p50, p95, p99).
3. Identify which endpoints or user flows are affected.
4. Correlate application metrics, logs and traces.
5. Check for downstream dependencies causing latency.
6. Compare performance across regions, instances and time periods.
7. Identify recent changes, traffic patterns or resource constraints.
8. Prioritize fixes based on impact to SLO and error budget burn rate.

2. Useful Commands (Monitoring & Tracing)

```
# Prometheus - check latency and error rates
> rate(http_request_duration_seconds_sum[5m]) / \
  rate(http_request_duration_seconds_count[5m])
> histogram_quantile(0.95, \
  rate(http_request_duration_seconds_bucket[5m]))
# Check error budget (example)
> (1 - (sum(rate(http_requests_total{status!~"5.."}[5m])) /
  sum(rate(http_requests_total[5m]))))
# Grafana - search for latency dashboard
# Query by endpoint or service
# Example using kubectl if metrics-server is available
$ kubectl top pods -n <ns>
$ kubectl get hpa -n <ns>
# Check traces (e.g., Jaeger, OpenTelemetry)
$ Search by trace ID or endpoint
```

3. Common Causes

- Slow database queries or external API calls.
- Inefficient application code or algorithms.
- Connection pool exhaustion.
- Inadequate caching strategy.
- Garbage collection or memory pressure.
- Network latency between services or regions.
- Increased traffic or inefficient resource usage.
- Synchronous calls to slow dependencies.
- Lack of timeouts, retries or circuit breakers.
- Under-provisioned service instances.

4. Fixes and Prevention

- Optimize slow queries and external calls.
- Implement caching and asynchronous processing.
- Increase or tune resource limits and autoscaling.
- Set and enforce timeouts, retries and circuit breakers.
- Use load testing to validate performance.
- Prioritize work based on customer impact and SLO.
- Continuously review and improve architecture.

5. Using Error Budgets for Prioritization

- Calculate current error budget burn rate.
- Identify which services or endpoints consume the most budget.
- Focus on fixes that reduce latency and prevent budget exhaustion.
- Balance reliability work with new feature development.
- Communicate risk when error budget is low.
- Use the error budget to justify engineering effort and timeline.

6. Performance Analysis Areas

- End-to-end request tracing and latency breakdown.
- Database and external service performance.
- Service-to-service communication latency.
- Resource utilization (CPU, memory, network, I/O).
- Traffic patterns and peak usage times.
- Regional or AZ-specific issues.
- Application profiling and code-level bottlenecks.

7. Best Practices

- Define clear SLOs and SLIs for key services.
- Monitor latency percentiles and error budget.
- Set actionable and well-tuned alerts.
- Investigate and fix root causes, not just symptoms.
- Automate performance testing in CI/CD.
- Conduct regular reliability reviews.
- Document incidents and lessons learned.
- Continuously optimize for cost, performance and reliability.

Database and Stateful Workload Reliability

VERIQTA

Scenario 29

Database connections become exhausted during peak traffic, causing failures across multiple microservices. How do you contain the incident and identify the cause?

1. Investigation Steps

1. Confirm the symptoms and scope (across services and regions).
2. Check current connection count and limits.
3. Identify which services are consuming connections.
4. Review application connection pool settings.
5. Check for long-running or idle connections.
6. Look for traffic spikes, retries or failing dependencies.
7. Validate database resource usage (CPU, memory, I/O).
8. Correlate with recent changes (deployments, config, traffic).

2. Useful Commands (PostgreSQL, MySQL examples)

```
# PostgreSQL
$ psql -U <user> -d <db>
$ SELECT count(*) FROM pg_stat_activity;
$ SELECT state, count(*) FROM pg_stat_activity
    GROUP BY state;
$ SELECT pid, usename, application_name, state,
    query_start, query FROM pg_stat_activity
    ORDER BY state, query_start;
$ SELECT * FROM pg_settings WHERE name =
    'max_connections';
$ SELECT * FROM pg_stat_activity WHERE state =
    'idle' ORDER BY state_change;
# MySQL
$ mysql -u <user> -p
$ SHOW STATUS LIKE 'Threads_connected';
$ SHOW VARIABLES LIKE 'max_connections';
$ SHOW PROCESSLIST;
$ SELECT user, host, db, command, time, state,
    info FROM information_schema.processlist;
```

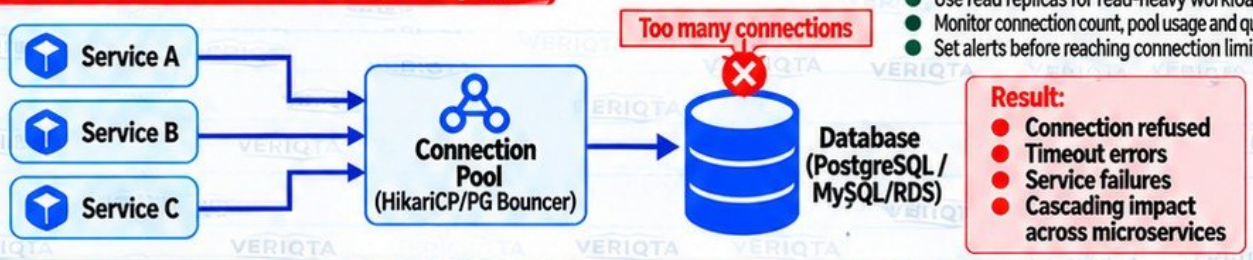
3. Common Causes

- Too many connections (hitting max limit).
- Application not using connection pooling.
- Leaked or idle connections.
- Long-running queries or locked transactions.
- Auto-scaling creating many short-lived connections.
- Retries causing connection storms.
- Misconfigured pool size or timeout.
- Database failover or restart not closing old connections.
- Network issues keeping connections open.
- Shared database used by multiple services without limits.

4. Fixes and Prevention

- Temporarily increase connection limit (if safe).
- Throttle or scale services generating excess traffic.
- Enable and tune connection pooling.
- Kill long-running or idle connections.
- Optimize slow queries and add indexes.
- Implement connection limits per service.
- Add circuit breakers and retry limits.
- Use read replicas for read-heavy workloads.
- Monitor connection count, pool usage and query time.
- Set alerts before reaching connection limits.

5. Database Connection Flow (Example)



Scenario 30

A production database failover completes, but applications remain unavailable. How would you investigate stale connections, DNS behavior, replication, and client recovery?

1. Investigation Steps

1. Confirm the failover status and new primary endpoint.
2. Check application errors and logs.
3. Verify DNS records and TTL.
4. Look for stale connections in applications and connection pools.
5. Validate replication status.
6. Confirm security groups and network connectivity to the new primary.
7. Check read/write endpoint configuration (RDS or self-managed).
8. Test database access from application hosts and recent deployments.

2. Useful Commands (AWS, DNS, Database)

```
# Check RDS/Aurora failover status
$ aws rds describe-db-instances \
  --db-instance-identifier <db-id> \
  --query 'DBInstances[0].DBInstanceStatus'\
$ aws rds describe-db-clusters \
  --db-cluster-identifier <cluster-id>
# Check DNS resolution
$ nslookup <db-endpoint>
$ dig <db-endpoint>
$ dig +trace <db-endpoint>
# Check replication (PostgreSQL example)
$ SELECT pg_is_in_recovery();
$ SELECT * FROM pg_stat_replication;
# Check active connections
$ SELECT * FROM pg_stat_activity;
# Test connectivity
$ nc -zv <db-endpoint> 5432
```

3. Common Causes

- Applications using stale connections.
- DNS caching (high TTL) pointing to old endpoint.
- Connection pool not refreshing after failover.
- Replication not fully caught up.
- Read/write endpoint misconfiguration.
- Security group or network ACL blocking access.
- Application hardcoded to old primary IP.
- Client libraries not handling failover properly.
- Stale DNS in container or VM resolver cache.
- Incomplete cluster recovery or promotion.

4. Fixes and Prevention

- Restart application pods/instances to clear pools.
- Reduce DNS TTL and use proper endpoints (RDS).
- Implement connection retry with failover logic.
- Verify replication and cluster health.
- Update security groups and network rules.
- Use RDS cluster endpoint (not instance endpoint).
- Enable automated failover and test regularly.
- Implement application-level health checks.
- Monitor failover events and client error rates.

5. Client Recovery Strategies

- Use short connection timeouts.
- Enable automatic reconnection and retries.
- Detect and drop stale connections.
- Implement exponential backoff.
- Use health checks to verify database readiness.
- Gracefully handle connection failures in application logic.

6. Monitoring and Alerting

- Track active connections and connection limit usage.
- Monitor replication lag and failover events.
- Alert on connection errors, timeouts and retries.
- Monitor DNS resolution and endpoint changes.
- Track application error rates and latency.
- Set SLOs for database availability and performance.

7. Best Practices

- Use managed database services (RDS/Aurora).
- Configure connection pooling and limits.
- Implement automatic failover and multi-AZ.
- Regularly test failover and recovery procedures.
- Monitor replication, connections and query performance.
- Keep runbooks updated and practice incident response.

Security and Secrets Management

VERIQTA

Scenario 31

A production secret is accidentally committed to a public Git repository. What immediate containment, credential rotation, and investigation steps are required?

1. Investigation Steps

1. Confirm what secret was exposed (type, scope, environment).
2. Check the commit history and time of exposure.
3. Identify which systems, services and users might be affected.
4. Determine if the secret has been used or abused.
5. Review repository visibility (public forks, clones, logs).
6. Check access logs, audit trails and unusual activity.
7. Identify blast radius across environments (dev, staging, prod).
8. Document timeline and evidence for post-incident review.

2. Useful Commands (Git, Cloud, Audit)

```
# View commit history for the file
git log --oneline -- <file>          # show commits

# Remove secret from history (use BFG or filter-repo)
git filter-repo --path <file> --invert-paths

# Check if repo is public
gh repo view <owner>/<repo> --json visibility

# AWS: check recent usage of an access key
aws iam get-access-key-last-used --access-key-id AKIA...

# AWS: rotate access key
aws iam create-access-key --user-name <user>
aws iam update-access-key --access-key-id <old-key> \
    --status Inactive

# Check CloudTrail for suspicious activity
aws cloudtrail lookup-events \
  --lookup-attributes AttributeKey=AccessKeyId,\
  AttributeValue=AKIA... \
  --max-results 50
```

3. Common Risks

- Unauthorized access using the exposed secret.
- Data theft, modification or deletion.
- Persistence via newly created credentials.
- Lateral movement to other services.
- Secrets leaked in build logs, containers or artifacts.
- Public forks retaining the secret.
- Automated scanners and attackers quickly using it.
- Reputation and compliance impact.

4. Fixes and Prevention

- Immediately revoke and rotate all exposed secrets.
- Invalidate sessions, tokens and keys.
- Remove the secret from Git history completely.
- Scan all repositories and artifacts for the secret.
- Review and tighten IAM permissions (least privilege).
- Enable secret scanning (GitHub, GitLab, etc.).
- Use secret management tools (AWS Secrets Manager, HashiCorp Vault, SSM Parameter Store).
- Implement pre-commit and CI/CD secret scanning.
- Use short-lived credentials and automatic rotation.
- Conduct a post-incident review and update processes.

5. Secret Exposure and Rotation Flow (Example)



Scenario 32

A container image passes vulnerability scanning but is compromised through its build pipeline. How would you strengthen supply-chain security and artifact trust?

1. Investigation Steps

1. Understand the attack path in the build pipeline.
2. Review CI/CD logs, build scripts and dependencies.
3. Identify compromised components (base images, dependencies, plugins).
4. Check for malicious code injection or script tampering.
5. Verify the integrity of artifacts and image signatures.
6. Review access controls for the pipeline (tokens, runners, secrets).
7. Check for unusual network activity during the build process.
8. Determine scope and any deployed impacted images.

2. Useful Commands (Docker, CI/CD, Verification)

```
# Inspect image details
docker image inspect <image>:<tag>

# Scan image for vulnerabilities (example)
trivy image <image>:<tag>

# Verify image signature (cosign)
cosign verify --key <public-key> <image>:<tag>

# Check build logs (GitHub Actions example)
gh run view <run-id> --log

# Check pipeline configuration
cat .github/workflows/<file>.yml

# List image layers and suspicious files
docker history <image>:<tag>
docker run --rm -it <image>:<tag> sh

# Check running containers and image digest
kubectl get pods -o wide
kubectl describe pod <pod-name> | grep -i image
```

3. Common Causes

- Compromised base image or dependency.
- Malicious code in build scripts or CI/CD pipeline.
- Insecure third-party plugins or actions.
- Stolen CI/CD tokens or credentials.
- Supply-chain attack on upstream package.
- Tampered artifacts or unsigned images.
- Lack of artifact integrity verification.
- Insufficient access controls on build environment.
- Simulated vulnerability scans that miss logic attacks.
- Persistence through poisoned images.

4. Fixes and Prevention

- Use trusted, minimal and regularly updated base images.
- Implement image signing and verification (cosign, Notary).
- Use Software Bill of Materials (SBOM) and dependency tracking.
- Enforce reproducible builds and immutable artifacts.
- Isolate and harden CI/CD runners (least privilege).
- Scan for secrets, malware and policy violations in build pipeline.
- Use provenance (SLSA) and attestation for artifacts.
- Restrict and audit access to CI/CD systems.
- Monitor for unusual build activity and network calls.
- Implement admission policies to verify image signatures.

5. Supply-Chain Security Flow (Simplified)



Distributed Systems and Cascading Failures

VERIQTA

Scenario 33

A downstream service slows down, triggering retries that overwhelm the entire platform. How would you investigate and control cascading failures?

1. Investigation Steps

1. Confirm the symptoms, scope and timeline.
2. Identify which service is slowing down and its error/latency rate.
3. Check retry configuration (retry count, backoff, timeout).
4. Analyze traffic amplification across services.
5. Review service dependencies and call graphs.
6. Check resource saturation (CPU, memory, threads, connections).
7. Look for circuit breakers, rate limits and bulkheads.
8. Use logs, traces and metrics to find the root cause.

2. Useful Commands (Kubernetes, Tracing)

```
# Check service latency and error rate
kubectl top pod -n <namespace>
kubectl logs <pod> --tail=100 | grep ERROR
# Check downstream service health
curl -w "time_total: %{time_total}\n" \
  https://downstream-service/api/health
# View traces (Tempo/Jaeger example using CLI)
curl -s "http://tempo:3200/api/search" \
  -G --data-urlencode 'q={service="api"}' \
  --data-urlencode 'limit=5'
# Check connection/threads (Java example)
jstack <pid> | grep -i "http"
# Check retry configuration (env or configmap)
kubectl describe deploy <service> | grep -i retry
```

3. Common Causes

- Aggressive retry logic with no backoff.
- Downstream service latency or partial failure.
- Timeouts set too high.
- Lack of circuit breakers or rate limiting.
- Connection pool exhaustion.
- Thread or worker exhaustion.
- Synchronous calls without bulkheads.
- Traffic amplification (many services retrying).
- Resource saturation (CPU, memory, network).
- Cascading failures across dependent services.

4. Fixes and Prevention

- Implement exponential backoff and jitter.
- Use circuit breakers and bulkheads.
- Set appropriate timeouts and retry limits.
- Add rate limiting and load shedding.
- Use asynchronous patterns where possible.
- Implement graceful degradation and fallbacks.
- Isolate dependencies and enforce SLIs/SLOs.
- Monitor dependencies and set alerts.

5. Cascading Failure Flow (Example)



Scenario 34

A message queue backlog grows rapidly while consumers appear healthy. How do you distinguish throughput limits, poison messages, consumer lag, and dependency saturation?

1. Investigation Steps

1. Confirm backlog growth rate and current queue depth.
2. Check consumer metrics (throughput, latency, error rate).
3. Verify if consumers are actually processing messages.
4. Look for poison messages or repeated failures.
5. Check downstream dependencies (databases, APIs, other services).
6. Analyze resource usage on consumers (CPU, memory, threads).
7. Review message processing time and concurrency settings.
8. Use logs, tracing and dead-letter queue (DLQ) to identify the cause.

2. Useful Commands (AWS, Kafka, RabbitMQ)

```
# Kafka: check consumer lag
kafka-consumer-groups.sh --bootstrap-server \
  <broker> --describe --group <group-name>
# Kafka: check topic details
kafka-topics.sh --bootstrap-server <broker> \
  --describe --topic <topic>
# RabbitMQ: check queue stats
rabbitmqctl list_queues name messages consumers
# AWS SQS: check queue attributes
aws sqs get-queue-attributes --queue-url <url> \
  --attribute-names ApproximateNumberOfMessages
  ApproximateNumberOfMessagesNotVisible
# Check consumer logs for errors
kubectl logs <consumer-pod> --tail=100 | grep ERROR
# Check message age (SQS example)
aws sqs get-queue-attributes --queue-url <url> \
  --attribute-names ApproximateAgeOfOldestMessage
```

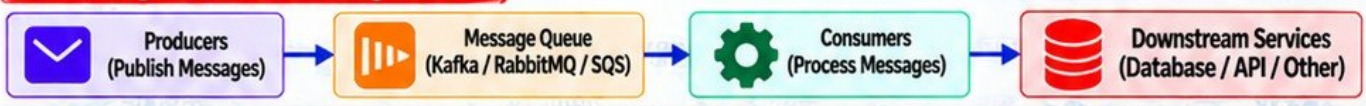
3. Common Causes

- Throughput limits (consumers too slow).
- Insufficient consumer concurrency.
- Poison messages causing repeated failures.
- Downstream service latency or unavailability.
- Incorrect message visibility timeout (SQS).
- Large messages or heavy processing time.
- Resource saturation on consumer instances.
- Network or cross-region latency.
- Misconfigured acknowledgement/commit.
- Consumer lag due to rebalancing or crashes.

4. Fixes and Prevention

- Increase consumer concurrency and scaling.
- Identify and handle poison messages (DLQ).
- Optimize message processing time.
- Set appropriate visibility timeouts and retries.
- Implement backpressure and rate limiting.
- Monitor queue depth, consumer lag and age.
- Use idempotent processing and retry logic.
- Ensure downstream dependencies are healthy.
- Set alerts for backlog growth and processing errors.

5. Message Queue Processing Flow



Platform Engineering and Developer Experience

Scenario 35

Development teams bypass the internal deployment platform because it slows delivery. How would you investigate adoption problems and redesign the platform without weakening governance?

1. Investigation Steps

1. Understand why teams are bypassing the platform.
2. Collect feedback (surveys, interviews, support tickets).
3. Measure deployment time and compare with alternative methods.
4. Identify specific pain points (complexity, slow approvals, poor documentation, reliability issues).
5. Review platform logs, metrics and failure rates.
6. Check governance requirements and compliance constraints.
7. Analyze team use cases and different workloads.
8. Identify quick wins and long-term improvements.

2. Useful Commands (Example: Kubernetes, Platform Tools)

```
# Check deployment events and failures
kubectl get events -A --sort-by=.lastTimestamp

# Check platform usage (example if using Argo CD)
argocd app list
argocd app history <app-name>

# Check CI/CD pipeline duration
kubectl get pipeline -A
kubectl get runs -A --sort-by=.status.startTime

# Analyze platform logs (example using Grafana/Loki)
{app="platform"} |= "error" | count_over_time[1h]

# Check resource usage on the platform
kubectl top nodes
kubectl top pods -A --sort-by=cpu

# Check audit logs for bypass methods
kubectl logs -n audit | grep "manual" | grep "deploy"
```

3. Common Causes

- Slow and complex deployment process.
- Too many manual approvals.
- Poor documentation and learning curve.
- Platform instability or frequent failures.
- Lack of useful features and self-service options.
- Rigid governance and security controls.
- Long feedback cycles from the platform team.
- Teams needing flexibility for experiments.
- Insufficient tooling integrations (e.g., Git, Slack).
- Perceived lack of ownership by developer teams.

4. Fixes and Prevention

- Simplify and automate the deployment process.
- Provide self-service and clear documentation.
- Improve platform reliability and performance.
- Implement progressive governance (guardrails).
- Offer reusable templates and golden paths.
- Add observability and fast feedback.
- Engage teams and establish a feedback loop.
- Support multiple use cases (dev, staging, prod).
- Measure and communicate platform value.
- Continuously improve based on real user needs.

5. Adoption Improvement Flow (Example)



Scenario 36

A shared Kubernetes platform serves dozens of teams, but one team's workloads affect everyone else. How would you design isolation, quotas, ownership, and operational boundaries?

1. Investigation Steps

1. Identify the affected services and scope of impact.
2. Check resource usage (CPU, memory, disk, network) per team, namespace and node.
3. Look for noisy neighbor workloads or misconfigured applications.
4. Review resource requests, limits and quotas.
5. Examine cluster events, logs and autoscaling behavior.
6. Check network policies and cross-namespace access.
7. Identify gaps in ownership and operational boundaries.
8. Analyze scheduling, storage and other shared dependencies.

2. Useful Commands (Kubernetes)

```
# Check namespace resource usage
kubectl top pods -A
kubectl top nodes
# Check resource quotas and limits
kubectl get resourcequota -A
kubectl describe resourcequota <namespace>
# Check pod resource requests and limits
kubectl get pods -A -o custom-columns=\
  NAMESPACE:.metadata.namespace,NAME:.metadata.name,\
  CPU-REQ:.spec.containers[*].resources.requests.cpu,\
  CPU-LIMIT:.spec.containers[*].resources.limits.cpu
# Check for noisy neighbors (node level)
kubectl describe node <node-name>
# Check network policies
kubectl get networkpolicy -A
kubectl describe networkpolicy <name>
# Check events and OOM kills
kubectl get events -A --sort-by=.lastTimestamp | grep -i "oom"
# Check storage usage
kubectl get pvc -A
kubectl top pvc -A (if metrics available)
```

3. Common Causes

- No resource limits or quotas per team.
- A team running resource-intensive workloads (noisy neighbor).
- Improper autoscaling configuration.
- Shared cluster resources without isolation.
- Lack of namespace or network isolation.
- Misconfigured storage or I/O-heavy workloads.
- Uncontrolled batch jobs or cron jobs.
- Shared dependencies (databases, caches).
- Lack of ownership and accountability.
- Insufficient monitoring and alerting.

4. Fixes and Prevention

- Implement namespace isolation per team.
- Set resource requests, limits and quotas.
- Use separate node pools for different workloads.
- Apply network policies and RBAC.
- Enforce storage and I/O limits.
- Implement fair cluster scheduling (taints/tolerations).
- Use rate limiting and pod disruption budgets.
- Establish clear ownership and operational boundaries.
- Monitor per-team resource usage and set alerts.
- Educate teams and provide platform guidelines.

5. Multi-Tenant Kubernetes Architecture (Example)



Disaster Recovery and Production Resilience

VERIQTA

Scenario 37

An entire cloud region becomes unavailable.
How do you decide whether to fail over, restore from backups, or wait for recovery?

1. Investigation Steps

1. Confirm the scope and impact of the region outage.
2. Check official cloud provider status and incident updates.
3. Identify affected services, applications and dependencies.
4. Verify multi-region architecture and current traffic routing.
5. Assess data consistency and replication status.
6. Check RTO/RPO requirements and business impact.
7. Evaluate failover readiness (DNS, load balancers, databases).
8. Decide: fail over, restore from backups, or wait based on evidence and risk.

2. Useful Commands (AWS examples)

```
# Check AWS region status
aws ec2 describe-regions --output table

# Check service health (AWS Health Dashboard)
aws health describe-events \
  --filter services=EC2,RDS,S3

# Check Route 53 health checks
aws route53 list-health-checks

# Check database replication status (RDS)
aws rds describe-db-instances \
  --query "DBInstances[*].[DBInstanceIdentifier,
  DBInstanceStatus,GlobalWriteForwardingStatus]"

# Check S3 cross-region replication (example)
aws s3api get-bucket-replication \
  --bucket my-bucket

# Check CloudFront origin health
aws cloudfront get-distribution --id <distribution-id>
```

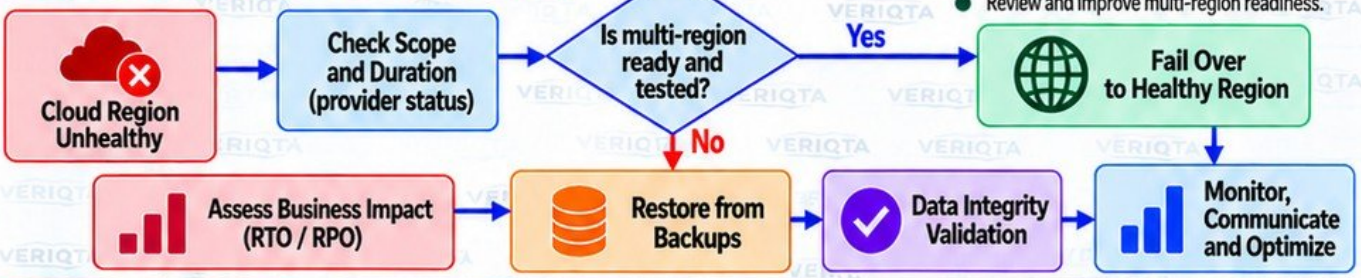
3. Decision Factors

- Scope and expected duration of the outage.
- Availability of healthy regions and capacity.
- Data replication lag and consistency.
- Business impact and customer requirements.
- RTO (Recovery Time Objective).
- RPO (Recovery Point Objective).
- Failover automation and tested runbooks.
- Cost, compliance and regulatory constraints.
- Risk of data loss or split-brain.
- Communication with stakeholders.
- Whether the issue is regional or global.

4. Options and Actions

- Fail over to a healthy region (if architecture supports it).
- Restore from backups (if no replication available).
- Wait for provider recovery (if lower business impact).
- Validate data integrity before serving traffic.
- Update DNS and traffic routing.
- Scale up resources in target region.
- Monitor closely and communicate clearly.
- Execute disaster recovery runbook.
- Review and improve multi-region readiness.

5. Regional Failover Decision Flow



Scenario 38

Backups report success, but a disaster recovery exercise reveals that critical data cannot be restored within the required recovery time.
What must change?

1. Investigation Steps

1. Confirm backup scope, frequency and retention.
2. Test the full restore process (not just backup success).
3. Measure actual restore time vs RTO.
4. Identify what is causing slow or failed restores (data size, dependencies, access).
5. Check backup integrity and completeness.
6. Review infrastructure and network constraints.
7. Validate application and data consistency after restore.
8. Document gaps and update DR strategy.

2. Useful Commands (Examples)

```
# List AWS Backup jobs and status
aws backup list-backup-jobs \
  --by-state COMPLETED --max-results 10

# Check last successful backup time
aws rds describe-db-instances \
  --query "DBInstances[*].[DBInstanceIdentifier,
  LatestRestorableTime]"

# Test restore in a staging environment (RDS example)
aws rds restore-db-instance-from-db-snapshot \
  --db-instance-identifier test-restore
  --db-snapshot-identifier <snapshot-id>

# Check EBS snapshots
aws ec2 describe-snapshots \
  --owner-ids self --query "Snapshots[*].[SnapshotId,
  StartTime,State,VolumeSize]"

# Check S3 backup objects
aws s3 ls s3://my-backup-bucket/ --recursive --human-readable
```

3. Common Causes

- Backups are incomplete or inconsistent.
- Restore process is slow or manual.
- Dependencies (networks, IAM, KMS) missing.
- Large data size and limited restore bandwidth.
- Snapshots need to be copied across regions.
- Lack of regular restore testing.
- Misconfigured backup retention or lifecycle.
- Application-specific recovery steps not documented.
- Hidden dependencies (databases, caches, queues).
- RTO/RPO not realistic for current architecture.

4. Fixes and Improvements

- Implement regular, full restore testing.
- Optimize backup strategy (incremental, snapshots).
- Use cross-region backups and replication.
- Automate the restore process with tested runbooks.
- Ensure all dependencies and IAM roles are included.
- Improve network bandwidth and storage performance.
- Review and meet realistic RTO/RPO targets.
- Document and test application recovery steps.
- Implement continuous backup monitoring and alerts.



Staff-Level Engineering Judgment and Leadership

VERIQTA

Scenario 39

Engineering leadership wants faster deployments, security wants stricter controls, and operations wants fewer incidents. How would you design a delivery strategy that balances all three?

1. Investigation Steps

1. Understand the goals, constraints and success metrics of each team.
2. Identify current delivery process, bottlenecks and pain points.
3. Review past incidents, security findings and compliance requirements.
4. Analyze risk profile and critical systems.
5. Collect data (lead time, change failure rate, incident rate, MTTR).
6. Evaluate existing automation, testing and observability.
7. Identify quick wins and long-term improvements.
8. Facilitate cross-team discussions to align on shared outcomes.

2. Useful Commands (Examples)

```
# Check deployment frequency (GitHub)
gh run list --limit 10
# Check change failure rate (example using logs)
kubectl get events -A --sort-by=.lastTimestamp |
grep -i failed
# Review incident history (example using metrics)
curl -s "http://prometheus/api/v1/query?query=
increase(incidents_total[30d])"
# Check security findings (example)
aws securityhub get-findings --max-results 10
# Check current pipeline performance
kubectl get pods -n ci --sort-by=.status.startTime
# Check cost impact of deployments (example)
aws ce get-cost-and-usage \
 --time-period Start=2026-10-01,End=2026-10-31 \
 --granularity DAILY --metrics UnblendedCost
```

3. Key Considerations

- Deployment speed and lead time.
- Security and compliance requirements.
- Change failure rate and incident volume.
- Risk tolerance for different environments.
- Automation, testing and quality gates.
- Observability and rollback capabilities.
- Cost impact and resource efficiency.
- Team productivity and developer experience.
- Regulatory and business constraints.
- Clear ownership and accountability.

4. Strategy and Solutions

- Implement automated CI/CD with security scanning and policy as code.
- Use progressive delivery (blue/green, canary).
- Enforce environment-specific controls (dev, test, prod).
- Implement automated testing and quality gates.
- Improve observability and incident response.
- Define clear change management and approval flow.
- Use metrics (DORA) to track and balance outcomes.
- Create shared goals and regular cross-team reviews.

5. Bailanced Delivery Strategy (Example)

Faster Deployments (Engineering)
- Automate CI/CD
- Self-service platform
- Reduce manual work
- Clear documentation

Stricter Controls (Security)
- Automated security scanning
- Policy as code
- Least privilege access
- Compliance checks

Fewer Incidents (Operations)
- Better observability
- Progressive delivery
- Rollback capability
- Incident response readiness

Shared Outcomes
- Fast, Safe and Reliable Deployments
- Reduced Risk
- Higher Team Productivity
- Better Customer Experience

Scenario 40

You inherit a fragile production platform with frequent outages, poor documentation, rising cloud costs, and no clear ownership. What would your first 90 days look like, and how would you measure improvement?

1. Investigation Steps

1. Understand the current state and key business impact.
2. Review recent incidents, timelines and root causes.
3. Assess architecture, documentation and ownership gaps.
4. Analyze cloud costs and resource usage.
5. Identify critical systems and dependencies.
6. Talk to stakeholders (engineering, product, operations, security).
7. Prioritize risks and quick wins.
8. Define measurable goals for 30/60/90 days.

2. Useful Commands (Examples)

```
# Check recent incidents (example using logs)
kubectl get events -A --sort-by=.lastTimestamp |
grep -i "error\|failed" | tail -20
# Check resource usage
kubectl top nodes
kubectl top pods -A --sort-by=cpu
# Check cloud costs (AWS)
aws ce get-cost-and-usage \
 --time-period Start=2026-10-01,End=2026-10-31 \
 --granularity DAILY --metrics UnblendedCost \
 --group-by Type=DIMENSION,Key=Service
# Find orphaned resources
aws resourcegroupstaggingapi get-resources \
 --tag-filters Key=Owner,Values=""
# Check documentation gaps (example)
rg -i "TODO|FIXME|missing" ./docs ./
```

3. Common Issues

- Frequent and recurring outages.
- Poor or outdated documentation.
- No clear ownership or accountability.
- High and growing cloud costs.
- Unmonitored systems and missing alerts.
- Manual and inconsistent processes.
- Security and compliance gaps.
- Technical debt and fragile architecture.
- Team burnout and lack of knowledge transfer.
- Unclear priorities and stakeholder misalignment.

4. First 90 Days Plan

- Days 1–30: Assess, stabilize and reduce risk.
- Days 31–60: Implement key improvements.
- Days 61–90: Scale, document and measure impact.
- Establish clear ownership and runbooks.
- Improve monitoring, alerting and incident response.
- Optimize cloud costs and right-size resources.
- Document architecture and standardize processes.
- Set measurable goals (e.g., availability, MTTR, cost reduction).

5. 90-Day Roadmap (Example)

Days 1–30 — Assess and Stabilize
- Understand current state
- Fix critical issues
- Implement basic monitoring
- Identify quick wins

Days 31–60 — Improve and Optimize
- Reduce incident frequency
- Optimize cloud costs
- Improve documentation
- Standardize processes

Days 61–90 — Scale and Measure
- Implement long-term improvements
- Establish clear ownership
- Measure and track KPIs
- Drive continuous improvement

Key Metrics to Track
- Availability (e.g., 99.9%)
- MTTR (Mean Time to Recover)
- Incident volume
- Deployment frequency
- Cloud cost reduction
- Documentation coverage